Catalogador de inventario

Asistente para catalogar el inventario de un hospital

Diseño técnico y diagramas UML

Arquitectura y diagramas de casos de uso, clases, secuencia, actividad (flujo), estados, componentes, despliegue y datos.

Versión	**3.1**
Fecha	**11 de diciembre de 2026**
Aplicación	**Catalogador 3.1 (Java 11 o superior · Windows, macOS y Linux)**
Documentos de partida	**Memoria del Proyecto · Información · 3 Excel de ejemplo**

Historial de versiones

1.0	16 de noviembre de 2026	Primera versión según la «Memoria del Proyecto».
2.0	24 de noviembre de 2026	Formato real de los Excel («Información»), catálogos GMDN y Mantenimiento, división por centro, 600.000 filas.
3.0	3 de diciembre de 2026	Modo manual o modo IA (servicios gratuitos, con versión gratuita y de pago), inventarios de prueba de 650.000 y 1.000.000 de filas.
3.1	11 de diciembre de 2026	Revisión final: adaptación a cada servicio de IA (ChatGPT, Gemini, Claude, Groq…) y a Ollama, correcciones para Windows, cómo conseguir las claves, pruebas documentadas y guía rápida.

Documentación del proyecto: Especificación de Requisitos (ERS) · Manual de usuario · Guía rápida de prueba.

Índice

1 Introducción 4

1.1 Propósito y alcance 4

1.2 Cómo leer los diagramas 4

1.3 Índice de diagramas 4

2 Arquitectura 5

2.1 Visión general 5

2.2 Diagrama de paquetes 6

2.3 Diagrama de componentes 7

2.4 Despliegue 8

2.5 Decisiones de diseño 9

3 Casos de uso 10

3.1 Casos de uso del modo IA 11

4 Diagramas de clases 12

4.1 Diccionario de clases 12

4.2 Vista general 14

4.3 Paquete modelo 15

4.4 Paquete logica (1/2): importación y catálogos 16

4.5 Paquete logica (2/2): memoria, persistencia y resultado 17

4.6 Paquetes excel y util 18

4.7 Paquete ia 19

4.8 Paquete ui 20

5 Diagramas de secuencia 21

5.1 Relación de diagramas de secuencia 21

5.2 Arranque y elección del modo de trabajo 22

5.3 Crear un proyecto desde el inventario 23

5.4 Buscar mientras se escribe 24

5.5 Calcular las sugerencias 25

5.6 Guardar una catalogación 26

5.7 Crear una entrada de Mantenimiento 27

5.8 Exportar el resultado 28

5.9 Aplicar un resultado anterior 29

5.10 Configurar el servicio de IA 30

5.11 Propuesta de la IA en segundo plano 31

5.12 Una consulta a la IA: adaptación, reintentos y errores 32

5.13 Pedir todas las propuestas y aceptar en bloque 33

6 Diagramas de actividad (flujo) 34

6.1 Flujo de trabajo completo 34

6.2 Revisar un grupo 35

6.3 Importación fila a fila 36

6.4 Consulta a la IA de un lote de grupos 37

6.5 Guardado continuo y recuperación 38

7 Diagramas de estados **39**
7.1 Grupo de equipos 39
7.2 Aplicación 40
7.3 Modo IA y propuesta de un grupo 41
8 Datos **42**
8.1 Ficheros que guarda el programa 42
9 Algoritmos **43**
9.1 Agrupación 43
9.2 Similitud para las sugerencias 43
9.3 Puntuación del buscador 43
9.4 Candidatos para la IA 43
9.5 Tamaño de cada consulta a la IA 44
10 Pruebas **45**

1 Introducción

1.1 Propósito y alcance

Este documento describe el diseño del Catalogador de inventario 3.1 mediante diagramas UML 2.5 —casos de uso, clases, secuencia, actividad (flujo), estados, componentes, despliegue y datos— acompañados de la explicación necesaria para entenderlos y mantener el programa. Complementa a la Especificación de Requisitos (qué debe hacer), al Manual de usuario (cómo se usa) y a la Guía rápida de prueba (cómo comprobar que cumple).

Novedades de la versión 3.1 en el diseño: el cliente de IA habla tres formatos (OpenAI, Anthropic y la API propia de Ollama) y se adapta a cada servicio y modelo (apartados 2.5, 4.7, 5.12 y 6.4); la confianza en los certificados del sistema (`Certificados`); y la clase `Ficheros`, que reúne lo que depende del sistema operativo (sustituir ficheros con reintentos, mensajes de error de disco, Escritorio de OneDrive, abrir carpetas y páginas web).

1.2 Cómo leer los diagramas

- Los nombres de clases, métodos y paquetes son los del código fuente (`codigo-fuente/src/main/java/catalogador`), en español.
- Visibilidad: `+` pública, `-` privada, `~` de paquete; subrayado o `{static}` = estático. Las clases anidadas se escriben `Externa.Interna`.
- Relaciones: composición (rombo negro), agregación (rombo blanco), asociación (flecha continua), dependencia (flecha discontinua), herencia (triángulo) y realización (triángulo discontinuo).
- En las secuencias, los números indican el orden de los mensajes; **alt**, **opt** y **loop** son fragmentos combinados (alternativa, opcional y repetición).
- Los diagramas anchos están en páginas apaisadas. Son vectoriales: en el PDF se pueden ampliar sin perder calidad.
- Las fuentes PlantUML (`.puml`) y los diagramas en SVG están en `documentacion/diagramas-uml`. Para regenerarlos: `java -jar plantuml.jar -charset UTF-8 -tsvg *.puml`.

1.3 Índice de diagramas

Fichero	Tipo	Contenido
d01, d01b	Casos de uso	Funciones del programa y del modo IA
d02, d02b, d17	Paquetes, componentes, despliegue	Arquitectura
d03 – d07, d20	Clases	Vista general y paquetes modelo, logica, excel/util, ui e ia
d10 – d14, d18, d21 – d26	Secuencia	Arranque, importar, buscar, sugerir, guardar, nueva entrada, exportar, resultado anterior, configurar la IA, propuestas de la IA, consulta a la IA, pedir todas y aceptar en bloque
d08, d09, d19, d27, d28	Actividad (flujo)	Flujo de trabajo, revisión de un grupo, importación fila a fila, consulta a la IA, guardado y recuperación
d15, d16, d29	Estados	Grupo, aplicación, modo IA y propuesta
d30	Datos	Ficheros guardados

2 Arquitectura

2.1 Visión general

El Catalogador es una aplicación de escritorio Java (Swing) organizada en capas, sin bibliotecas externas. Cada paquete tiene una responsabilidad:

Paquete	Responsabilidad	Depende de
`ui`	Ventanas y diálogos; atajos; tareas largas en segundo plano con avance; ajuste a la pantalla.	logica, ia, modelo, excel, util
`ia`	Modo IA: servicios, configuración, cliente HTTP adaptado a cada servicio, certificados, elección de candidatos, propuestas.	logica (Catalogos, Memoria), modelo, util
`logica`	Reglas de negocio: importar, buscar, sugerir, guardar, exportar.	modelo, excel, util
`modelo`	Entidades: proyecto, grupo, término oficial, estados.	util
`excel`	Lectura y escritura de .xlsx y .csv en flujo.	Java SE (StAX, ZIP), util
`util`	Texto (normalización, distancia), TSV, diarios, JSON y ficheros (lo que depende del sistema operativo).	Java SE

2.2 Diagrama de paquetes

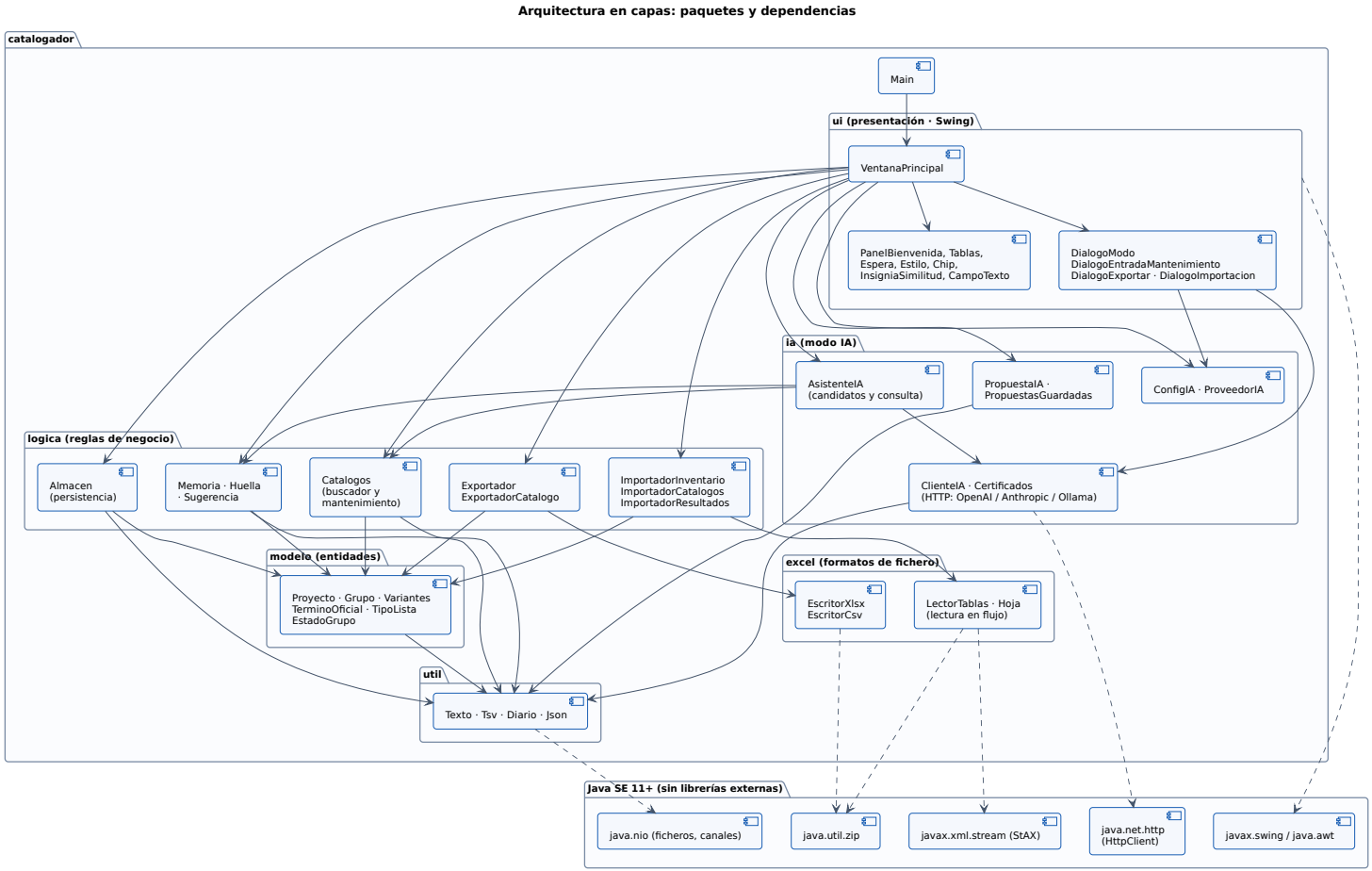


Figura 1. Arquitectura en capas: paquetes y dependencias.

2.3 Diagrama de componentes

Los componentes se comunican mediante interfaces pequeñas: `LectorTablas.Receptor` (recibe cada fila leída), `LectorTablas.Progreso` (informa del avance a la interfaz) y `EscritorXlsx.Filas` (genera cada fila de salida al escribirla, sin tener todo el fichero en memoria). El asistente de IA habla con los servicios por tres interfaces HTTP/JSON: la de OpenAI (que usan casi todos), la de Anthropic y la de Ollama.

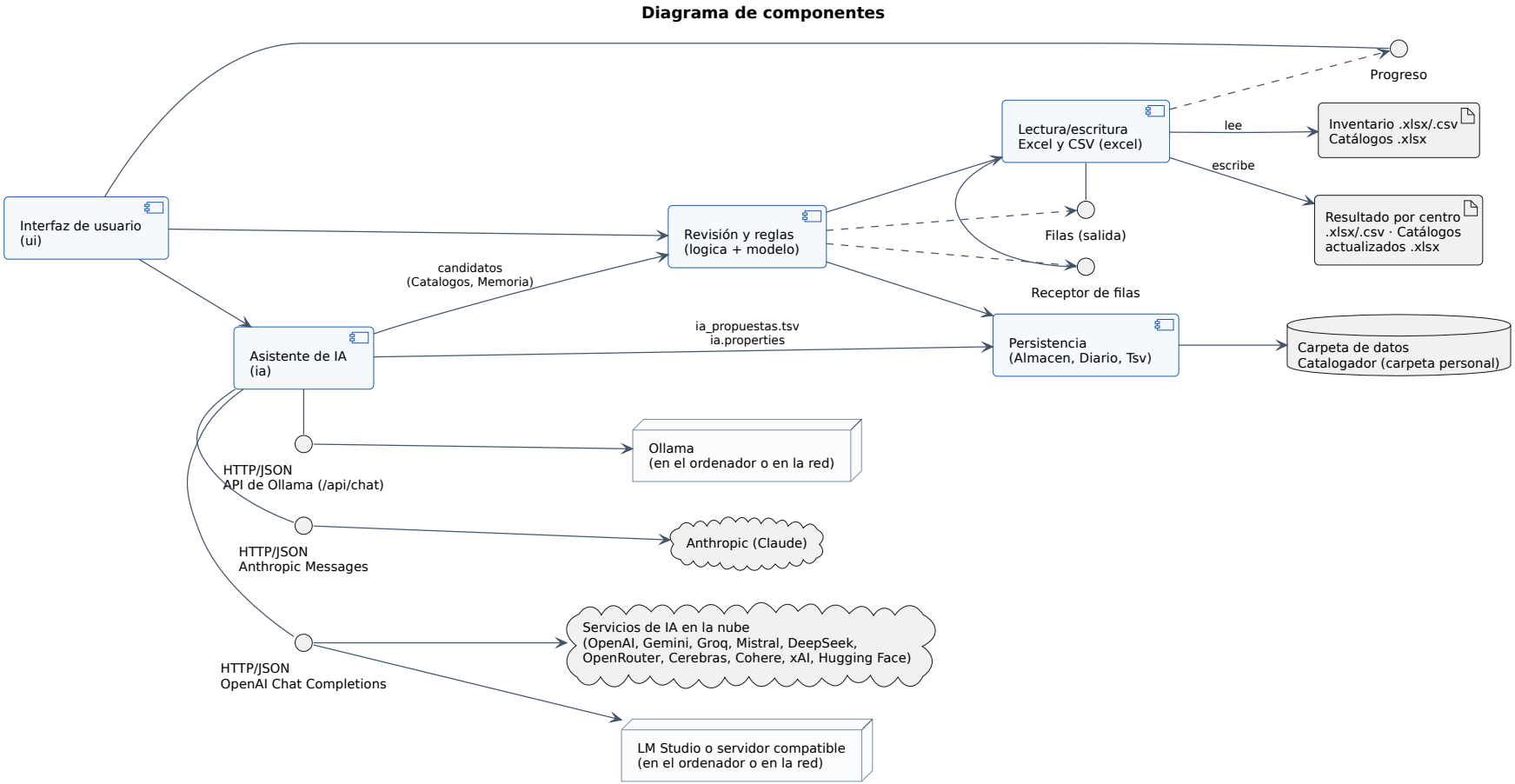


Figura 2. Diagrama de componentes.

2.4 Despliegue

Un único artefacto (`Catalogador.jar`) que se ejecuta en la máquina virtual de Java del ordenador del usuario, lanzado por un script para cada sistema que reserva hasta el 60 % de la memoria. Los datos se guardan en la carpeta `Catalogador` del usuario. En modo IA se añade la conexión con el servicio elegido: en la nube por HTTPS (con el proxy y los certificados del sistema), en el propio ordenador (Ollama, LM Studio, sin proxy) o en un servidor de la red del hospital.

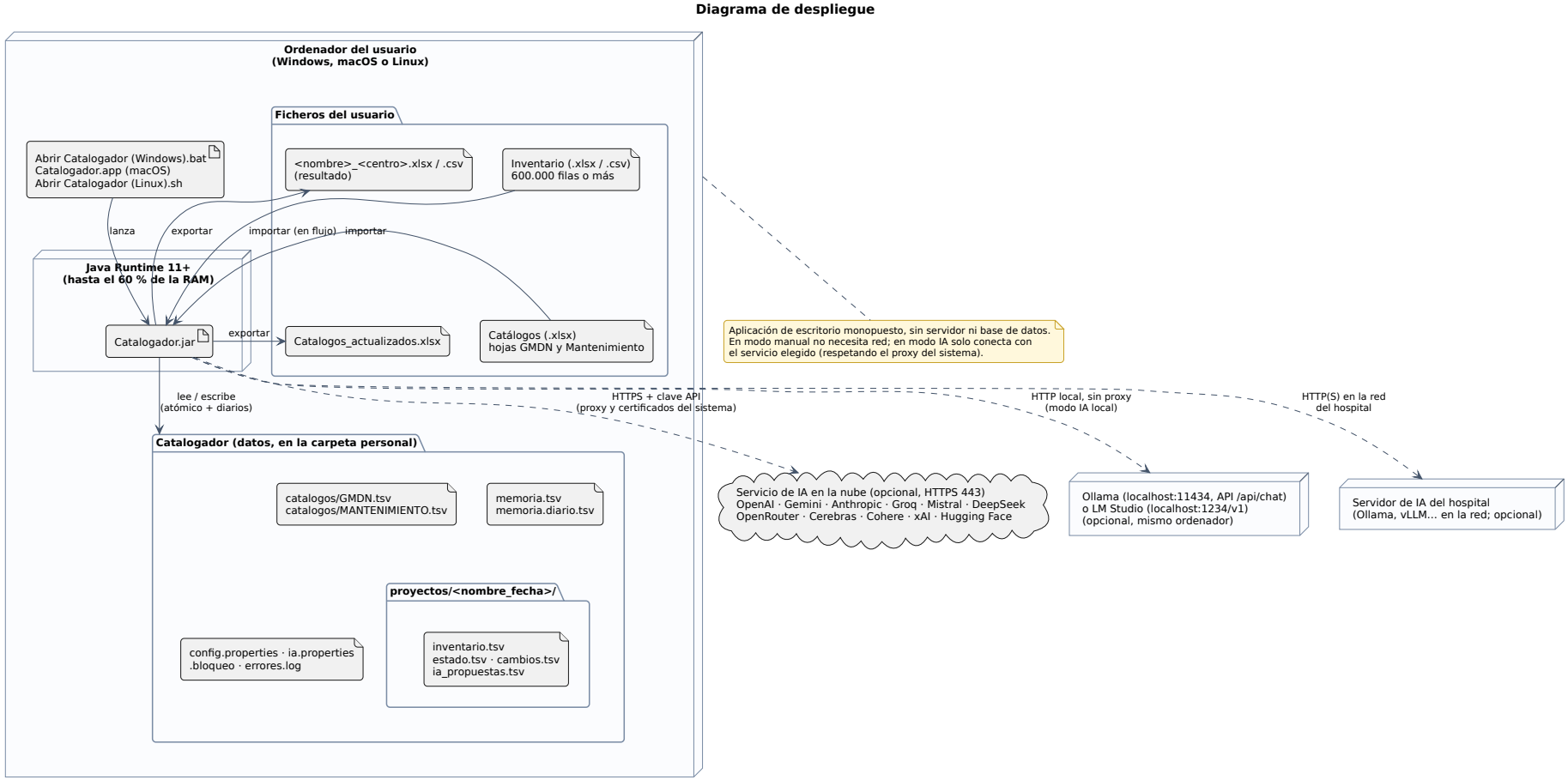


Figura 3. Diagrama de despliegue.

2.5 Decisiones de diseño

Decisión	Motivo
Java SE 11 sin bibliotecas externas	Un solo .jar que funciona en Windows, macOS y Linux sin instalar nada más que Java; sin dependencias que actualizar ni licencias que revisar.
Lectura de .xlsx en flujo con StAX	Un inventario de 1.000.000 de filas se lee fila a fila sin cargar el XML entero (Apache POI en modo normal necesitaría varios GB).
Proyecto en columnas y textos compartidos	Un array por dato y una sola copia de cada texto repetido: 1.000.000 de equipos ocupan unos 100 MB.
Diarios de solo añadir + instantáneas atómicas	Guardar cada decisión cuesta lo mismo con cualquier tamaño (menos de 1 ms) y un cierre brusco nunca deja ficheros corruptos. En Windows, la sustitución atómica se reintenta si el antivirus o el indexador tienen el fichero abierto un instante.
Búsqueda en un hilo aparte con «generaciones»	La interfaz no se bloquea al teclear; los resultados de búsquedas ya superadas se descartan.
Índice de trigramas en la memoria	Las sugerencias tardan menos de 2 ms de media con 30.000 clasificaciones.
La IA elige entre candidatos del catálogo	Evita que invente términos o códigos; reduce el texto enviado (y el coste); permite validar cada respuesta.
Consultas a la IA por lotes y por delante de la persona	Varios grupos por consulta (8; 4 con Groq y los locales) reducen el número de llamadas y los límites por minuto; la propuesta suele estar lista al llegar al grupo.
Tres formatos de API y adaptación por servicio	Casi todos los servicios usan el formato de OpenAI, pero cada uno admite opciones distintas (salida estructurada, razonamiento, límite de salida, temperatura). Anthropic tiene el suyo y Ollama, con su API propia, permite fijar el contexto y apagar el razonamiento. Lo que un servicio rechaza se quita y se aprende; lo que no cabe se parte.
Certificados de Java + del sistema	En las redes de los hospitales es habitual un proxy que inspecciona el tráfico cifrado con un certificado propio, instalado en Windows o macOS pero no en Java.
Propuestas guardadas en el proyecto	No se repite el gasto ni la espera si se cierra el programa.

3 Casos de uso

Los casos de uso (CU-01 a CU-18) están especificados en la ERS, capítulo 7. Aquí se muestran sus diagramas.

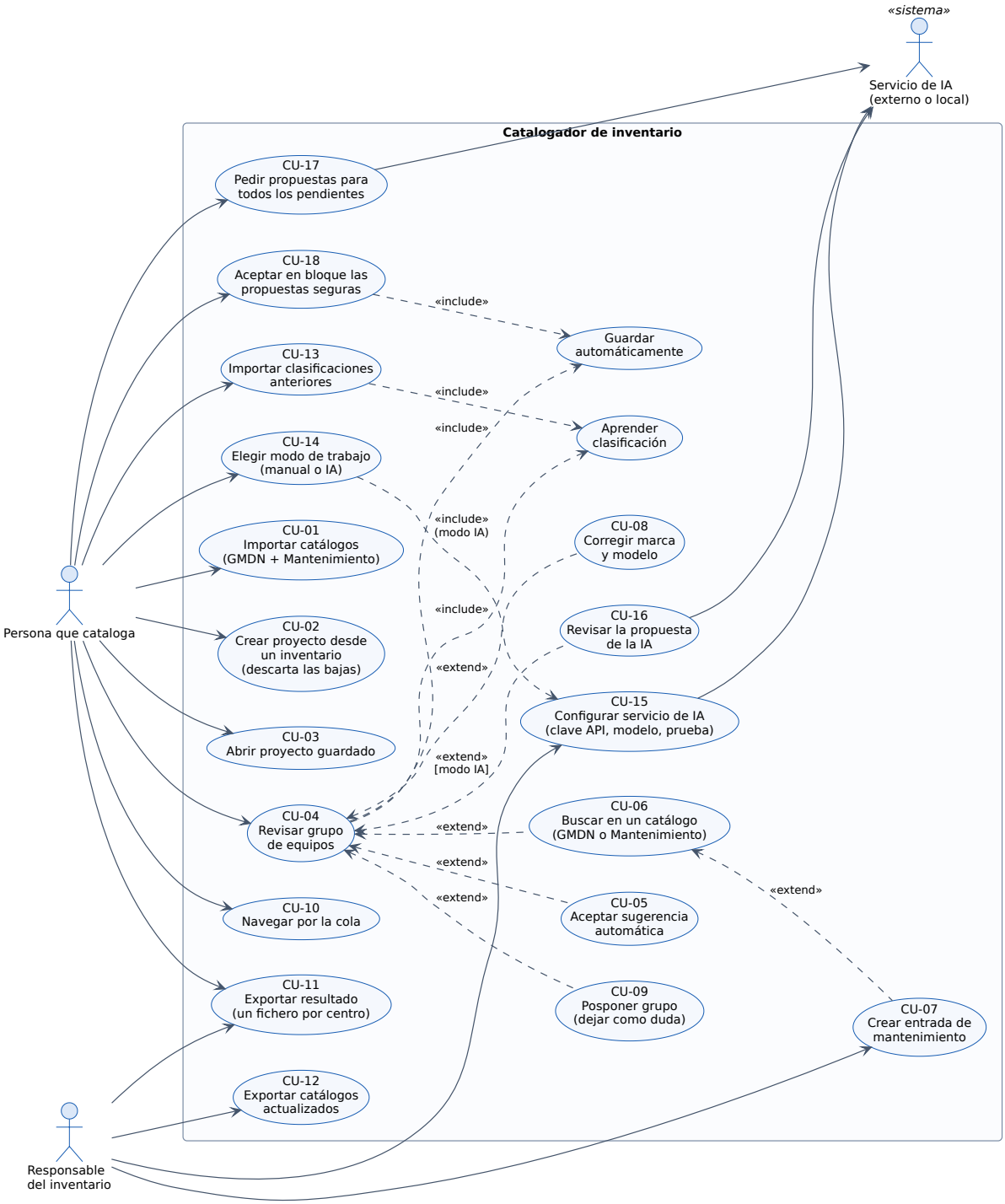


Figura 4. Casos de uso del Catalogador.

3.1 Casos de uso del modo IA

El servicio de IA es un actor de tipo sistema: solo responde a las consultas del programa. La persona elige el modo y el servicio (CU-14, CU-15, con los pasos para conseguir la clave), revisa las propuestas (CU-16) y puede pedirlas para todo el inventario (CU-17) y aceptarlas en bloque (CU-18).

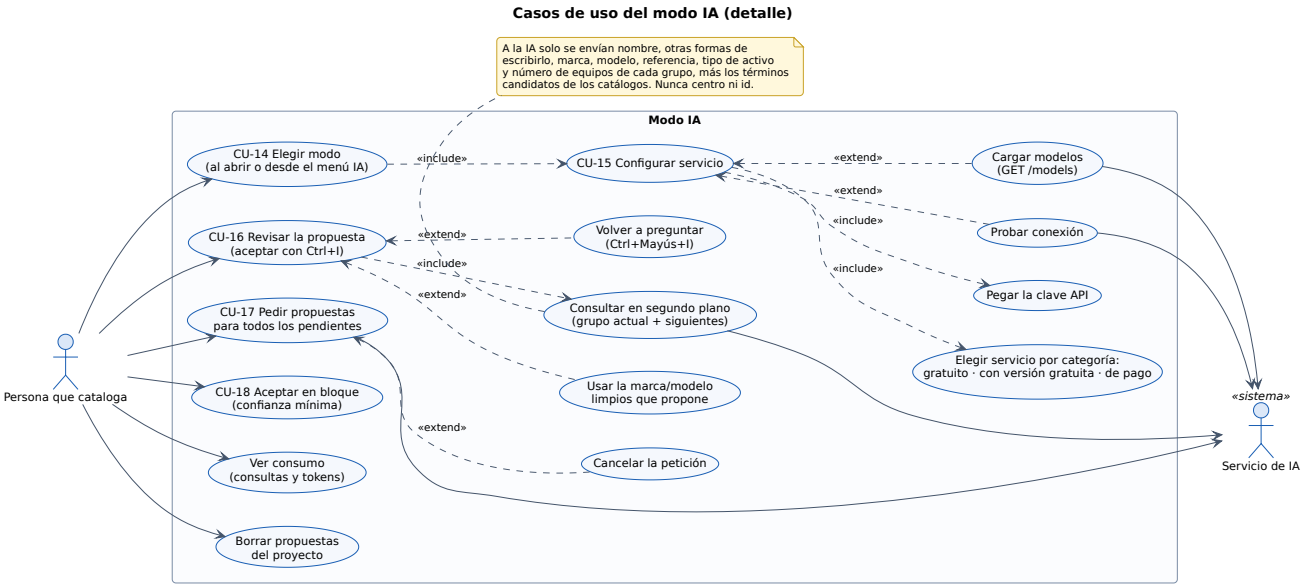


Figura 5. Casos de uso del modo IA en detalle.

4 Diagramas de clases

Primero, la responsabilidad de cada clase; después, los diagramas (vista general y uno por paquete), en páginas apaisadas.

4.1 Diccionario de clases

Clase	Responsabilidad
catalogador	
`Main`	Punto de entrada: proxy del sistema, aspecto visual, bloqueo de instancia, registro de errores; abre la ventana.
catalogador.modelo	
`Proyecto`	Equipos del inventario (por columnas), estadísticas de importación y grupos.
`Grupo`	Equipos iguales y resultado de su revisión (estado, término, correcciones, fecha, origen).
`Variantes`	Formas distintas de un dato del grupo con su número de apariciones.
`TerminoOficial` (+ `Nivel`)	Entrada GMDN o de Mantenimiento con sus datos originales y normalizados.
`TipoLista`, `EstadoGrupo`	Catálogo (GMDN, MANTENIMIENTO) y estado del grupo (PENDIENTE, REVISADO, POSPUESTO).
catalogador.logica	
`ImportadorInventario` (+ `Mapeo`)	Detecta las columnas del inventario y lo importa descartando las bajas.
`ImportadorCatalogos` (+ `Resultado`)	Lee el Excel de catálogos y valida sus hojas.
`ImportadorResultados` (+ `Resumen`)	Aplica un resultado anterior al proyecto (RN-10).
`Catalogos` (+ `Catalogo`, `Busqueda`, `Resultado`)	Catálogos cargados, búsqueda puntuada, códigos y entradas de Mantenimiento.
`Memoria` (+ `Entrada`), `Huella`	Clasificaciones y correcciones aprendidas; similitud e índice de trigramas.
`Sugerencia`	Propuesta con porcentaje, color y origen (memoria, IA o ambas); combinación memoria + IA.
`Almacen` (+ `InfoProyecto`)	Carpetas de datos, guardado y apertura de proyectos, configuración, bloqueo.
`Exportador` (+ `Opciones`, `Fichero`), `ExportadorCatalogo`	Resultado por centro (con nombres de fichero únicos aunque los centros solo se distingan por mayúsculas); catálogos con el formato original.
catalogador.ia	
`ProveedorIA` (+ `Categoria`, `Protocolo`, `FormatoJson`)	Servicios de IA admitidos: categoría, formato de API y de respuesta, dirección, modelos, página y pasos para conseguir la clave, lote y límite de entrada.
`ConfigIA` (+ `Modo`)	Modo de trabajo y configuración de cada servicio; claves y variables de entorno.
`ClienteIA` (+ `Respuesta`, `ErrorIA`, `InfoOllama`)	Peticiones HTTP en el formato de cada servicio, adaptación de opciones, reintentos y esperas, errores y contadores de uso.
`Certificados`	Contexto TLS que confía en los certificados de Java y en los del sistema (Windows, llavero de macOS).

Clase	Responsabilidad
`AsistenteIA` (+ `Candidato`, `Resultado`)	Candidatos, consulta por lotes (partiéndolos si no caben), validación y segunda vuelta.
`PropuestaIA`, `PropuestasGuardadas`	Propuesta para un grupo; diario de propuestas del proyecto.
catalogador.excel	
`LectorTablas` (+ `Receptor`, `Progreso`), `Hoja`	Lectura de .xlsx (ZIP + StAX) y .csv en flujo; vista previa de las primeras filas.
`EscritorXlsx` (+ `HojaSalida`, `Filas`, `Celda`), `EscritorCsv`	Escritura en flujo con estilos, filtros y anchos; CSV UTF-8 con marca.
catalogador.util	
`Texto`, `Tsv`, `Diario`, `Json`	Normalización y comparación de textos; ficheros TSV atómicos; diarios; JSON (con extracción del JSON de una respuesta con texto alrededor).
`Ficheros`	Lo que depende del sistema: sustituir un fichero con reintentos, mensajes claros de error de disco, abrir carpetas y páginas web, Escritorio (también en OneDrive).
catalogador.ui	
`VentanaPrincipal`	Ventana de trabajo y coordinación de todas las acciones; menú de la aplicación en macOS.
`DialogoModo`	Elección de modo manual o IA y configuración del servicio (con los pasos para conseguir la clave).
`DialogoImportacion`, `DialogoExportar`, `DialogoEntradaMantenimiento`	Elección de columnas; opciones de exportación; crear o corregir entradas.
`PanelBienvenida`, `Tablas`, `Espera`, `Estilo`, `Chip`, `InsigniaSimilitud`, `CampoTexto`	Pantalla inicial; modelos y dibujo de tablas; tareas largas; colores, fuentes y ajuste de las ventanas a la pantalla; etiquetas, insignia del porcentaje y campo con texto de ayuda.

4.2 Vista general

`VentanaPrincipal` coordina el trabajo: tiene el almacén de datos, los catálogos, la memoria, el proyecto abierto y el grupo actual, y en modo IA el asistente y las propuestas guardadas del proyecto. La lógica no conoce a la interfaz.

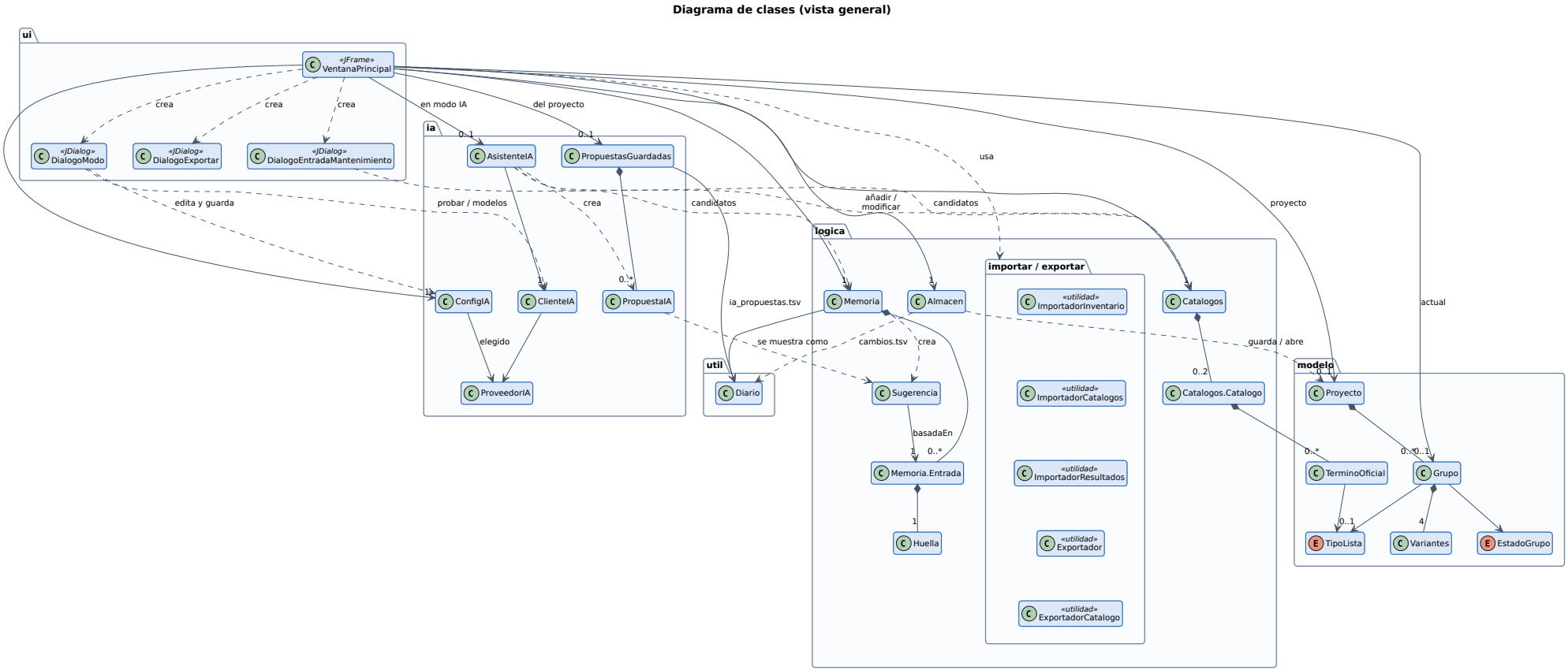


Figura 6. Vista general de las clases principales.

4.3 Paquete modelo

`Proyecto` guarda los equipos por columnas (un array por dato) y comparte los textos repetidos; `agrupar()` calcula la clave de cada equipo (RN-01), crea los `Grupo` y los ordena de más a menos equipos. Cada grupo lleva cuatro `Variantes` (nombre, marca, modelo y referencia con sus recuentos) y el resultado de la revisión. `TerminoOficial` representa una entrada de cualquiera de los dos catálogos, con sus datos originales (para exportarlos igual) y versiones normalizadas precalculadas para buscar deprisa.

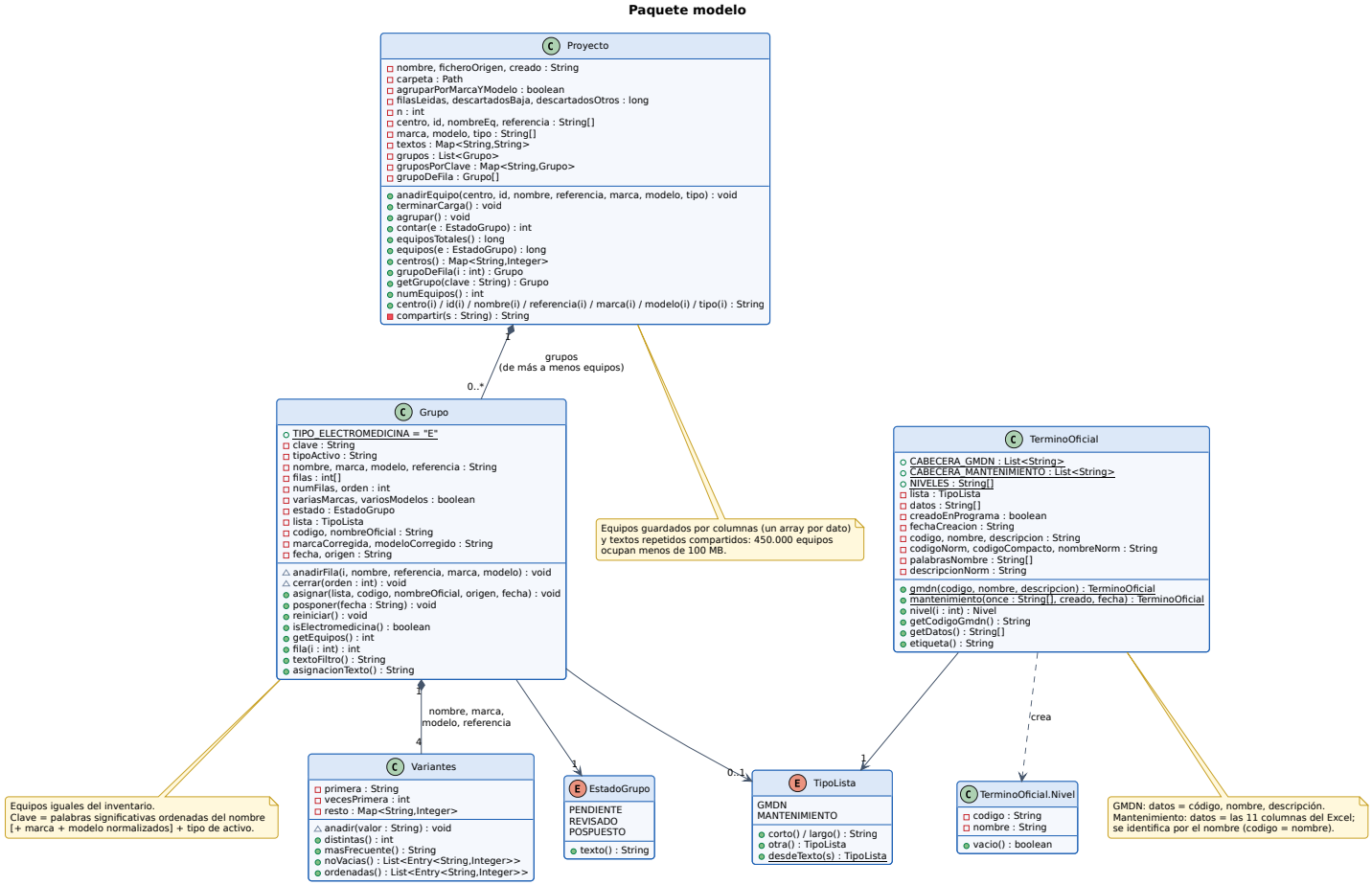


Figura 7. Paquete modelo.

4.4 Paquete logica (1/2): importación y catálogos

Los importadores leen los ficheros a través de `LectorTablas`. `Catalogos` mantiene los dos catálogos en un mapa `volatile` que se sustituye entero al cambiar, para que el hilo del buscador nunca vea un estado a medias; además busca, propone códigos y crea o corrige entradas de Mantenimiento.

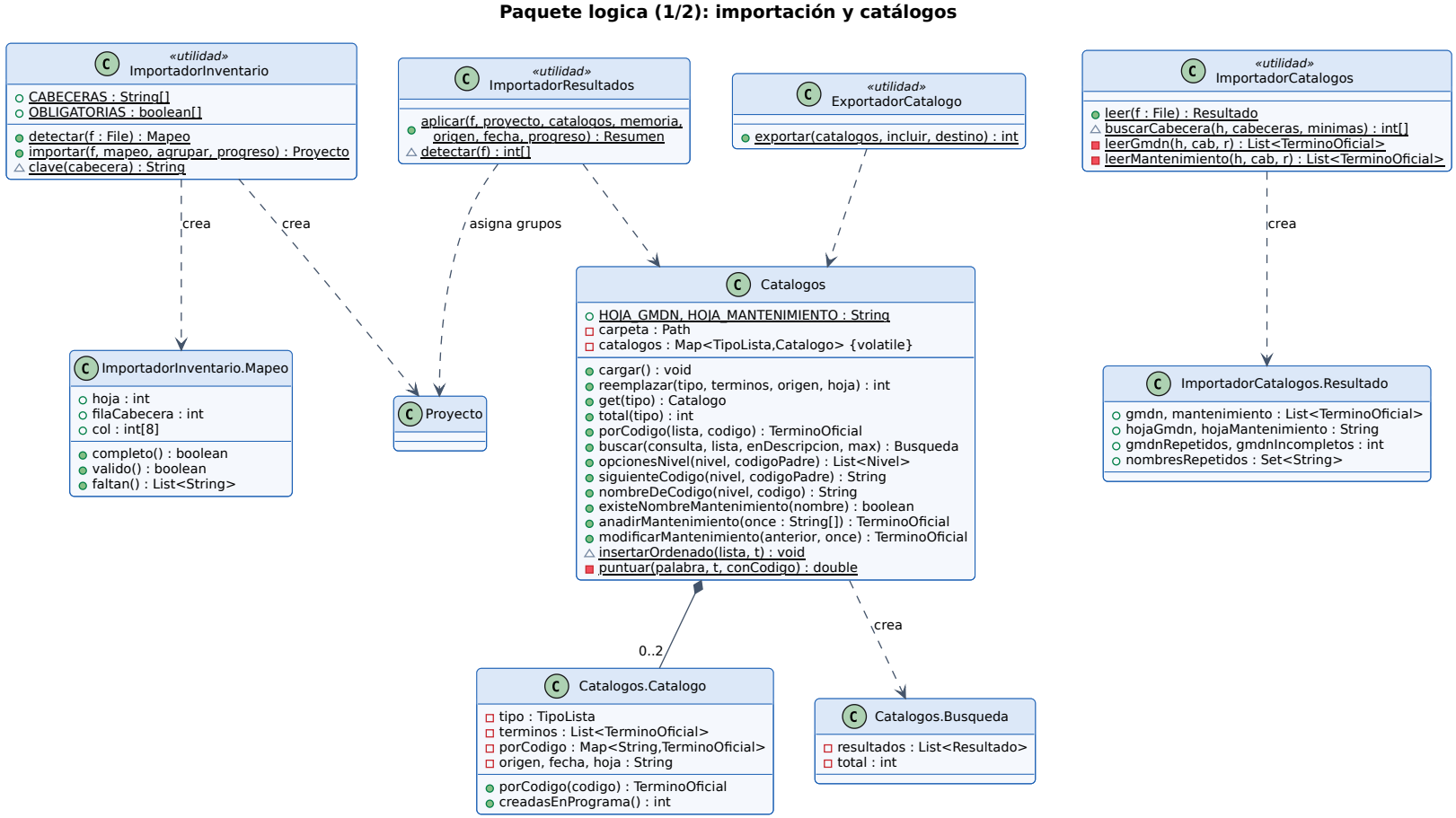


Figura 8. Paquete logica: importación y catálogos.

4.5 Paquete logica (2/2): memoria, persistencia y resultado

`Memoria` guarda lo aprendido (con su diario) y calcula las sugerencias con `Huella` (palabras y trigramas del nombre). `Sugerencia` indica su origen (memoria, IA o ambas) y sabe combinarse con la propuesta de la IA. `Almacen` guarda y abre los proyectos; `Exportador` genera los ficheros por centro.

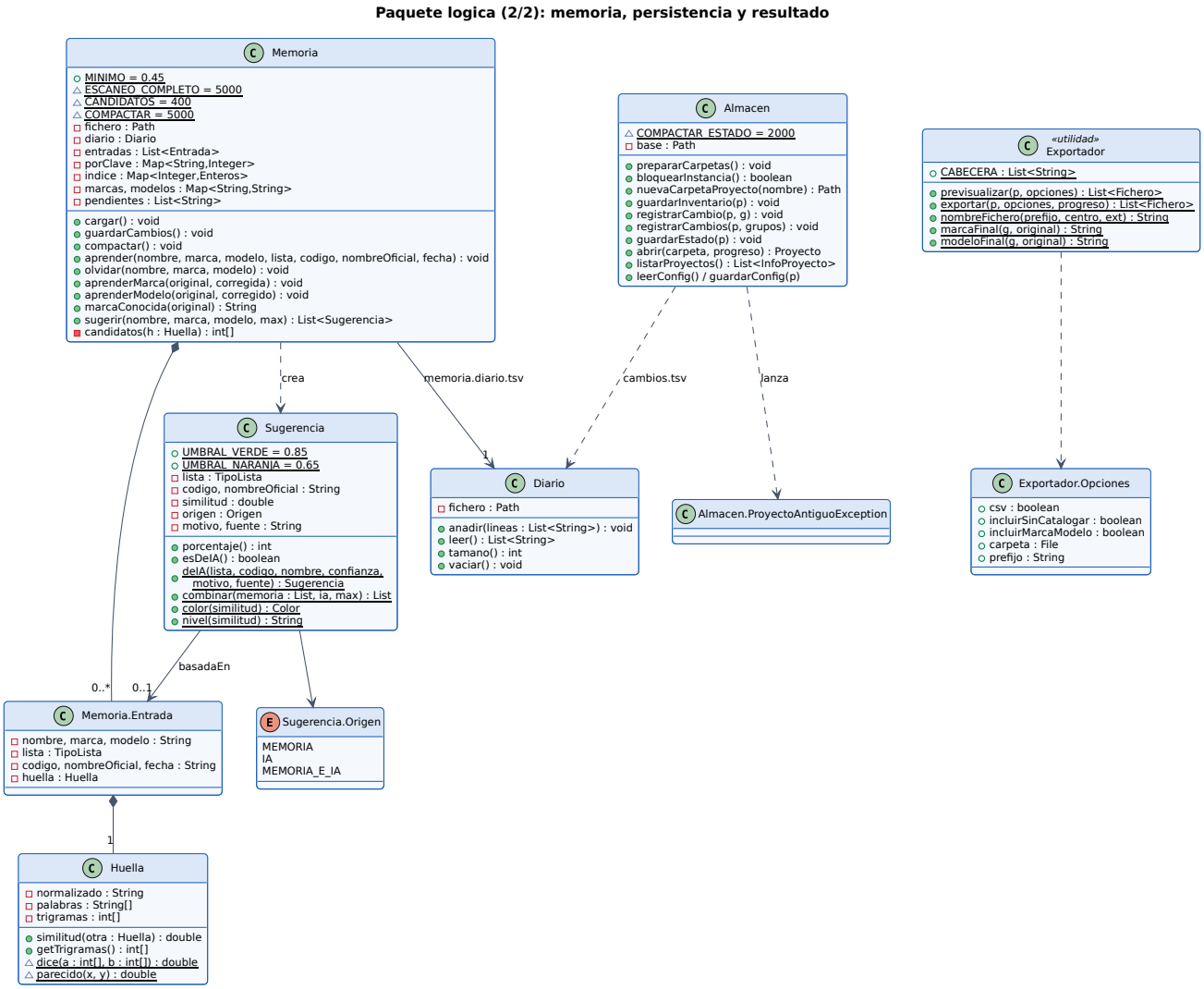


Figura 9. Paquete logica: memoria, persistencia y resultado.

4.6 Paquetes excel y util

`LectorTablas` entrega las filas una a una a un `Receptor`; `EscritorXlsx` y `EscritorCsv` piden cada fila a un objeto `Filas` al escribirla. En util, `Texto` normaliza y compara textos (incluida la limpieza de espacios invisibles), `Tsv` y `Diario` guardan los datos, `Json` lee y escribe JSON para las API de IA y `Ficheros` reúne lo que depende del sistema operativo.

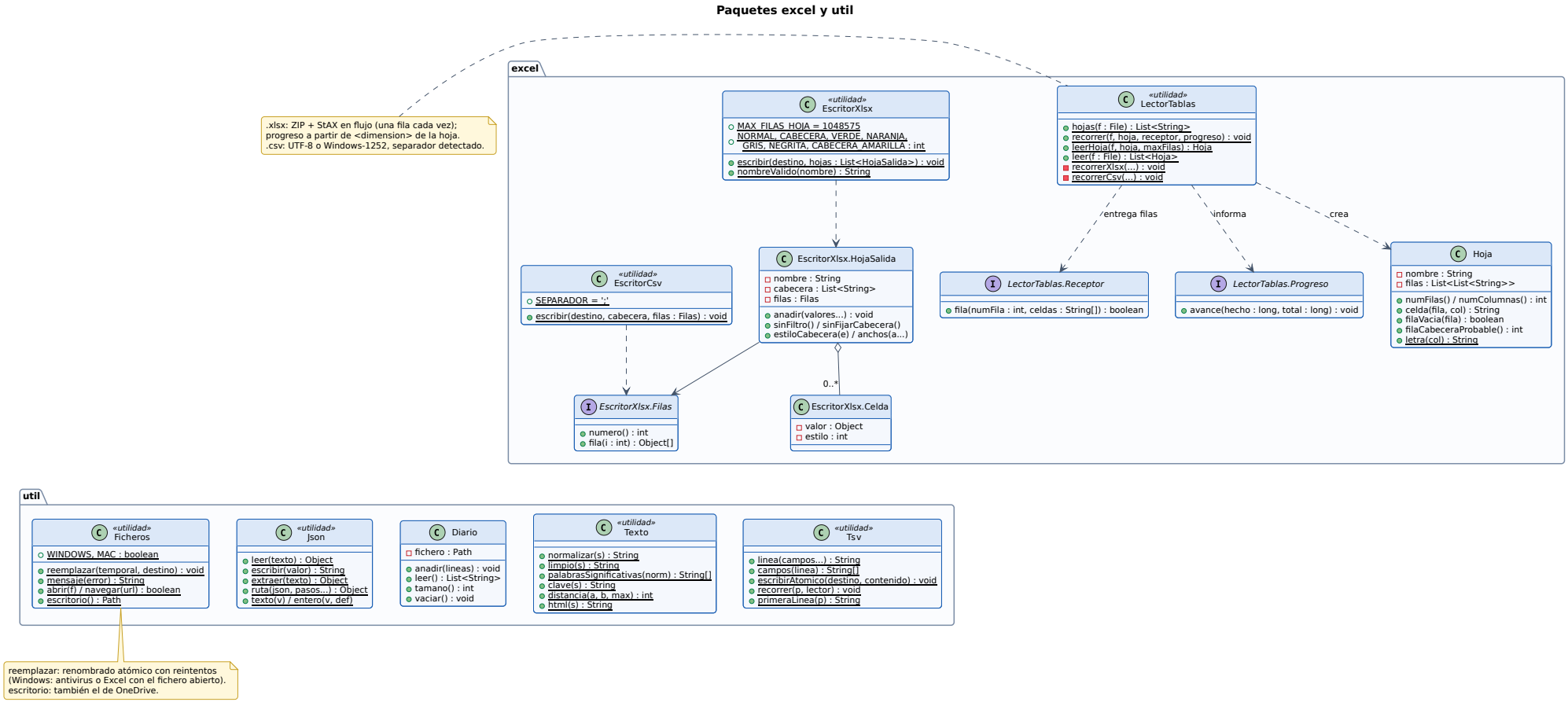


Figura 10. Paquetes excel y util.

4.7 Paquete ia

`ProveedorIA` describe cada servicio (categoría, formato de API y de respuesta, dirección, modelos, dónde y cómo se consigue la clave, lote y límite de entrada). `ConfigIA` guarda el modo y, por servicio, la clave, el modelo y la dirección en `ia.properties`. `ClienteIA` compone la petición en el formato del servicio, quita las opciones que el servicio rechaza, reintenta con la espera indicada y traduce los errores (`ErrorIA`); para Ollama lee antes las capacidades del modelo (`InfoOllama`). `AsistenteIA` elige los candidatos, compone la consulta (partiéndola si no cabe), valida la respuesta y crea las `PropuestaIA`, que `PropuestasGuardadas` conserva en el proyecto.

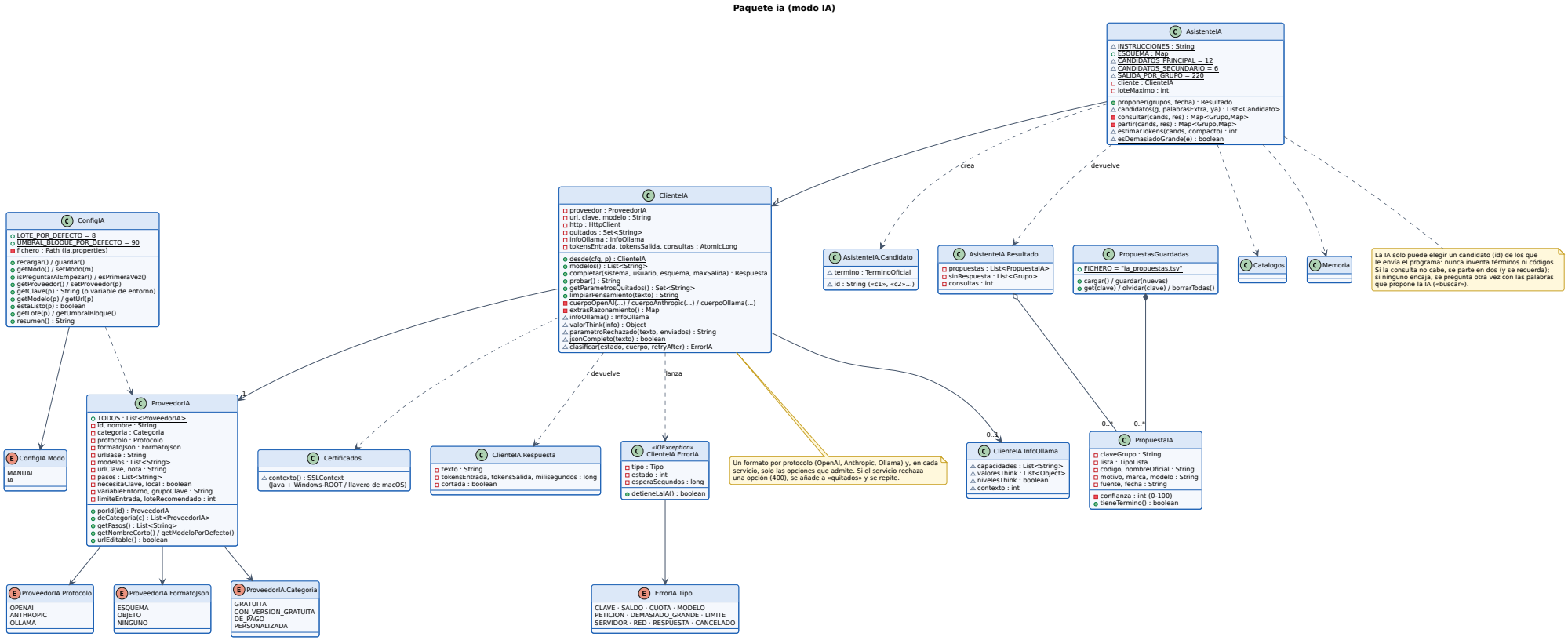


Figura 11. Paquete ia.

4.8 Paquete ui

La ventana principal y los diálogos heredan de las clases de Swing. `DialogoModo` es la ventana «¿Cómo quieres trabajar?». `Espera` ejecuta las tareas largas en un `SwingWorker` mostrando el avance (y, si procede, un botón Cancelar). `Estilo` ajusta cada ventana a la zona útil de la pantalla.

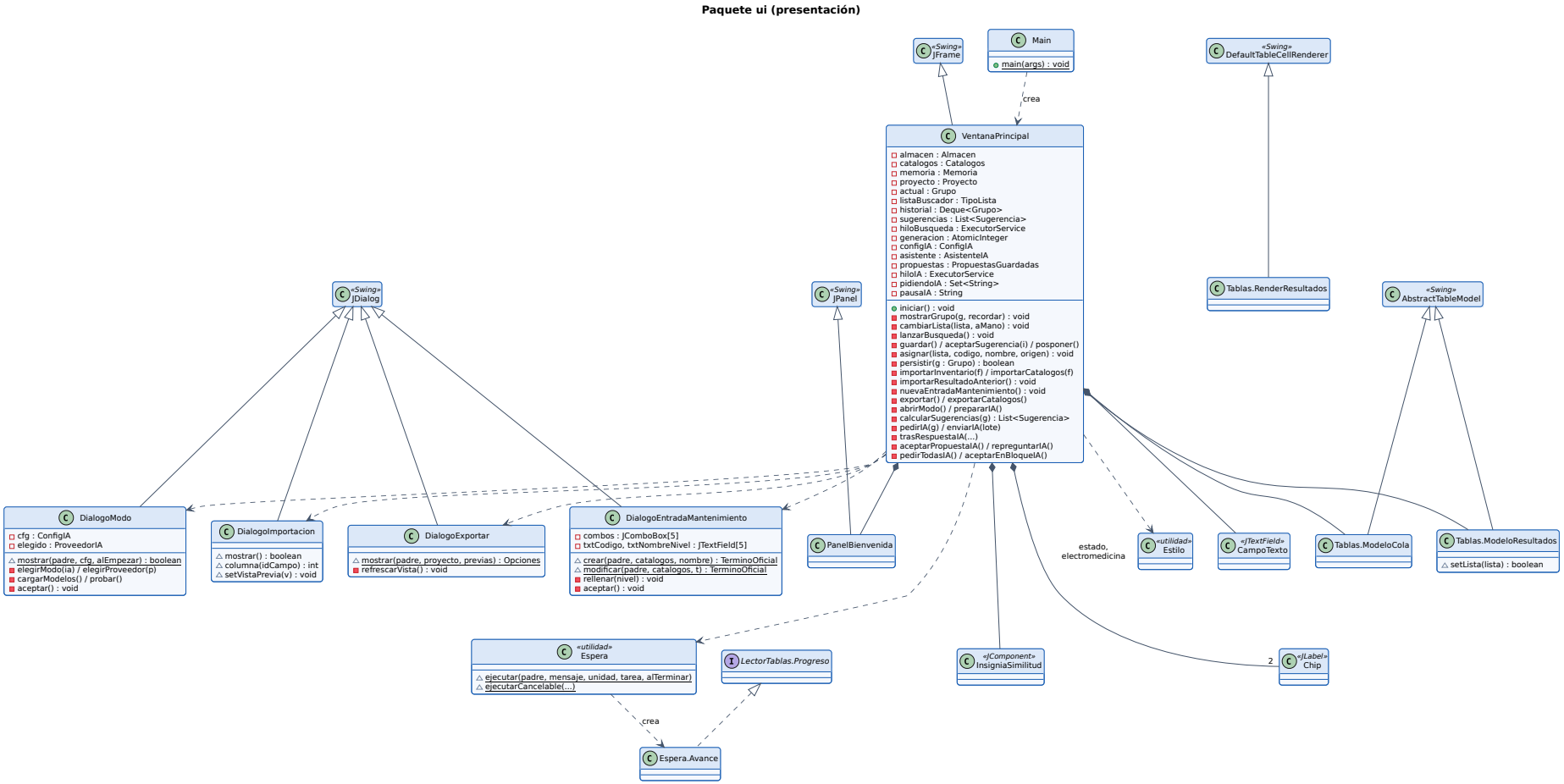


Figura 12. Paquete ui.

5 Diagramas de secuencia

Cada diagrama muestra los objetos que intervienen en una operación y los mensajes que se intercambian, en orden. Los nombres de los métodos son los del código fuente. Los diagramas grandes van en páginas apaisadas.

5.1 Relación de diagramas de secuencia

Apartado	Qué muestra	Casos de uso
5.2	Arranque y elección del modo de trabajo	CU-03, CU-14
5.3	Crear un proyecto desde el inventario (detección de columnas, lectura en flujo, agrupación)	CU-02
5.4	Buscar en el catálogo mientras se escribe	CU-06
5.5	Calcular las sugerencias de la memoria y combinarlas con la propuesta de la IA	CU-04, CU-05, CU-16
5.6	Guardar una catalogación (asignar, aprender, diarios)	CU-04, CU-05
5.7	Crear una entrada en el catálogo de Mantenimiento	CU-07
5.8	Exportar el resultado por centro	CU-11
5.9	Aplicar el resultado de una catalogación anterior	CU-13
5.10	Configurar el servicio de IA y probar la conexión	CU-15
5.11	Propuesta de la IA en segundo plano para el grupo que se ve y los siguientes	CU-16
5.12	Una consulta a la IA: formato del servicio, adaptación, reintentos y errores	CU-15 a CU-17
5.13	Pedir propuestas para todos los pendientes y aceptarlas en bloque	CU-17, CU-18

5.2 Arranque y elección del modo de trabajo

El lanzador abre `Main`, que configura el proxy del sistema y el aspecto, bloquea la carpeta de datos y crea la ventana. La carga de catálogos, memoria y último proyecto se hace en segundo plano. Después, si procede, se pregunta el modo; finalmente `prepararIA()` crea el asistente si el modo es IA y el servicio está listo.

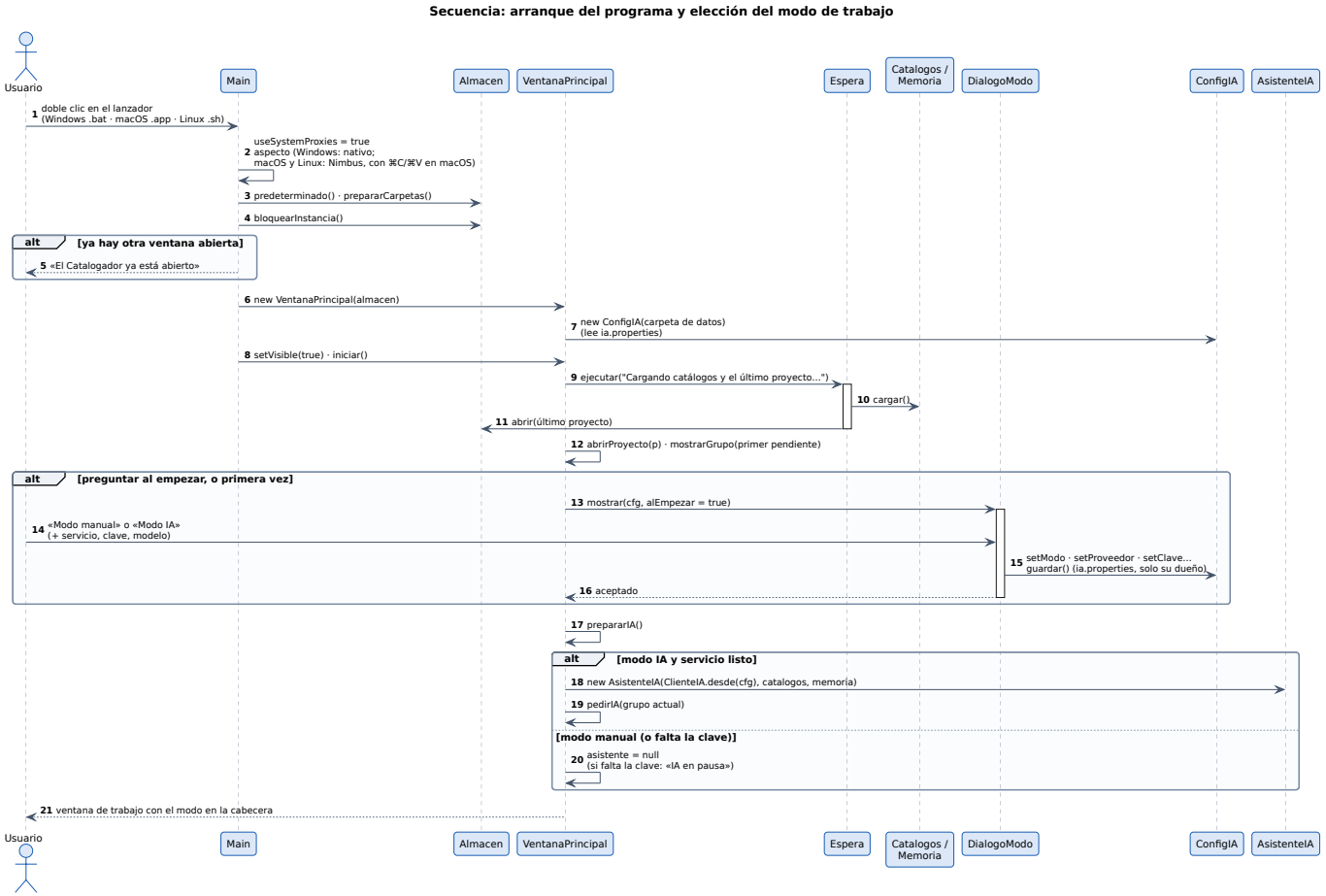


Figura 13. Secuencia de arranque.

5.3 Crear un proyecto desde el inventario

Primero se detectan las columnas (leyendo solo las primeras filas); tras la confirmación, el fichero se recorre una vez en flujo, contando las bajas y añadiendo los equipos; al final se agrupa y se guarda el proyecto.

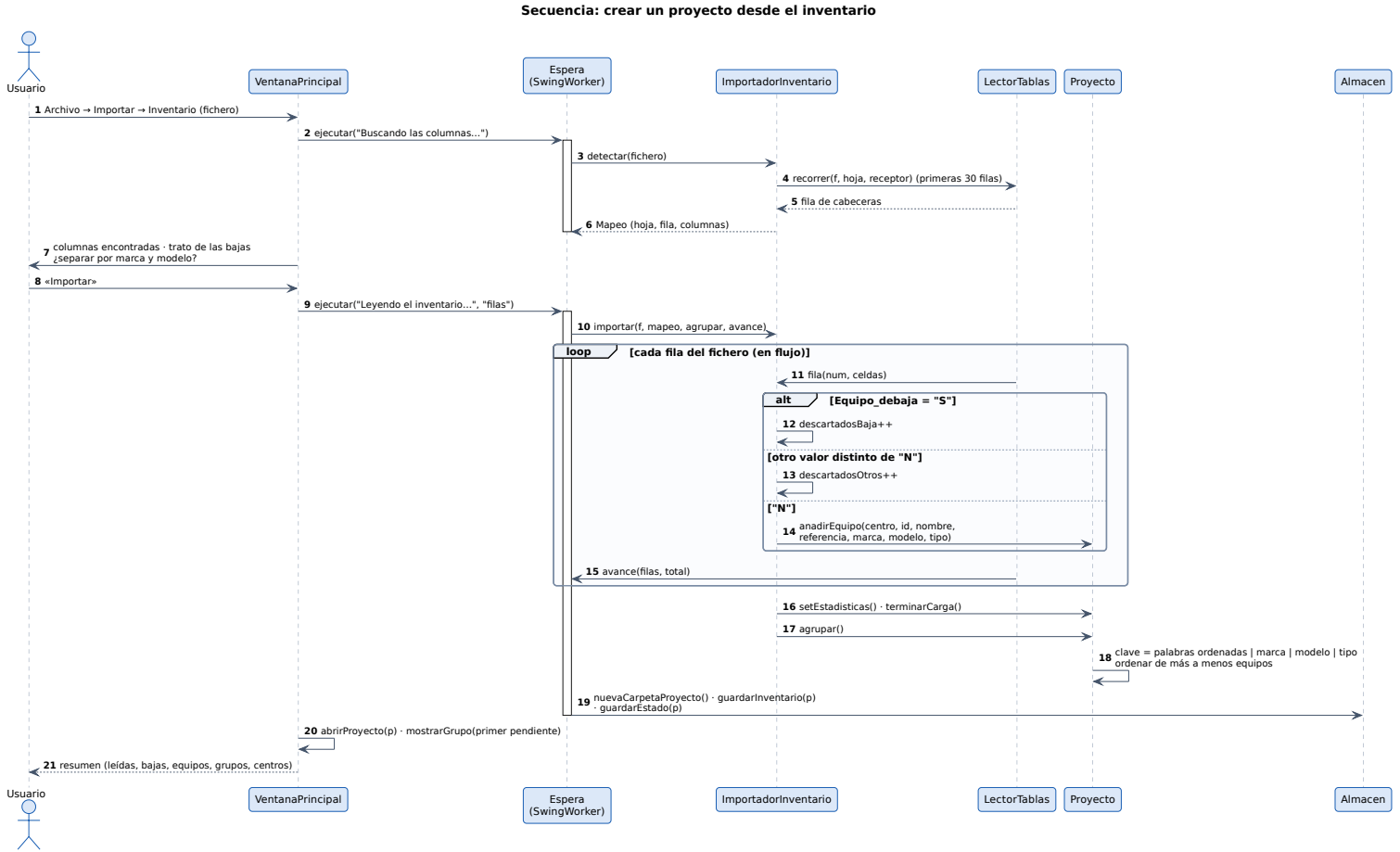


Figura 14. Secuencia: crear un proyecto desde el inventario.

5.4 Buscar mientras se escribe

Cada tecla reinicia un temporizador de 110 ms; al vencer, la búsqueda se envía al hilo del buscador con un número de «generación». Si al volver ya hay una búsqueda más reciente, el resultado se descarta.

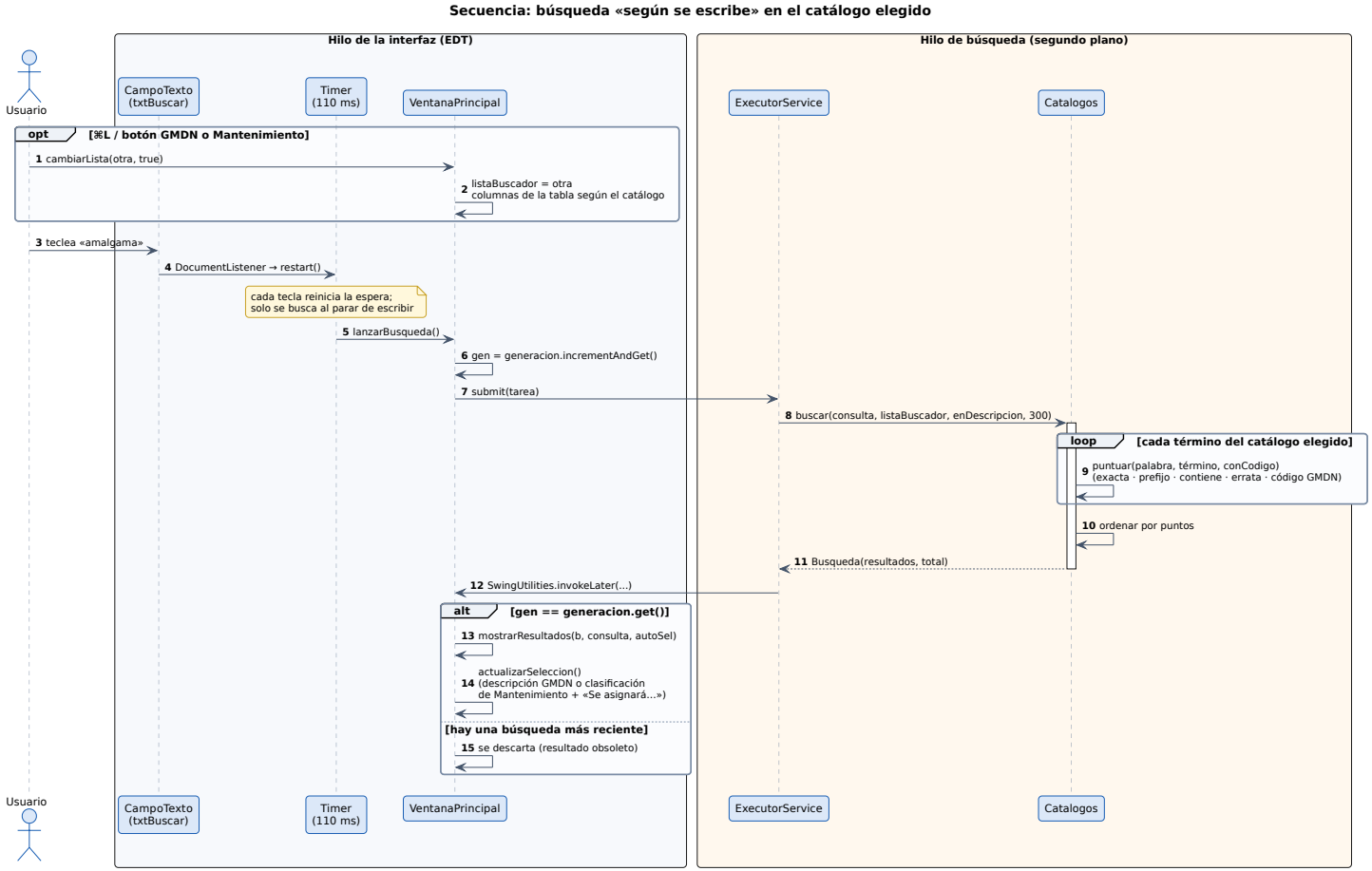


Figura 15. Secuencia: buscar en el catálogo mientras se escribe.

5.5 Calcular las sugerencias

Con memorias grandes, el índice de trigramas preselecciona 400 candidatas antes de calcular la similitud completa. En modo IA, la propuesta guardada del grupo se combina con las sugerencias (RN-15).

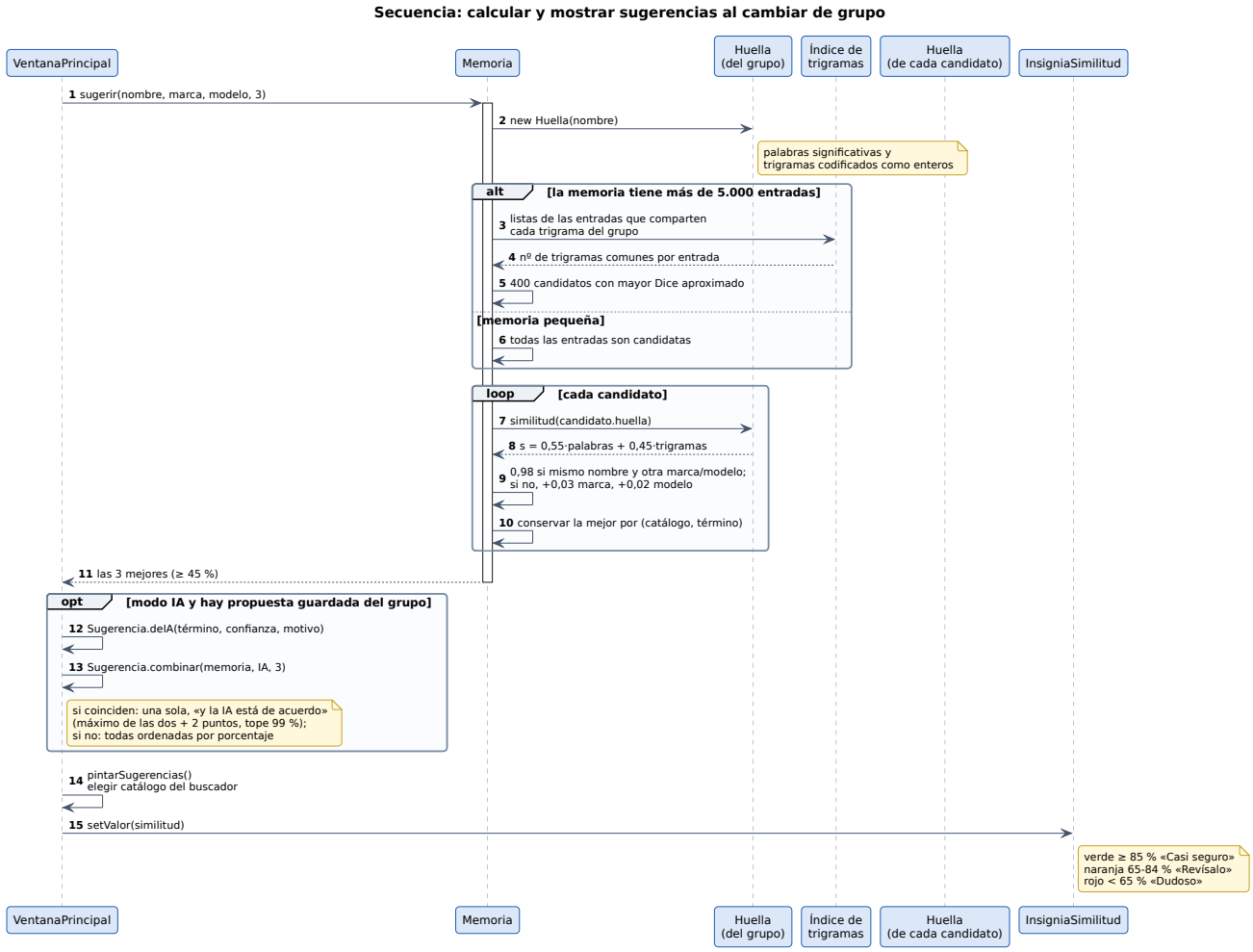


Figura 16. Secuencia: sugerencias de la memoria y de la IA.

5.6 Guardar una catalogación

Asignar, aprender y anotar el cambio en los diarios (forzados a disco). Cada 2.000 cambios se escribe una instantánea completa; cada 5.000 líneas de memoria, se compacta.

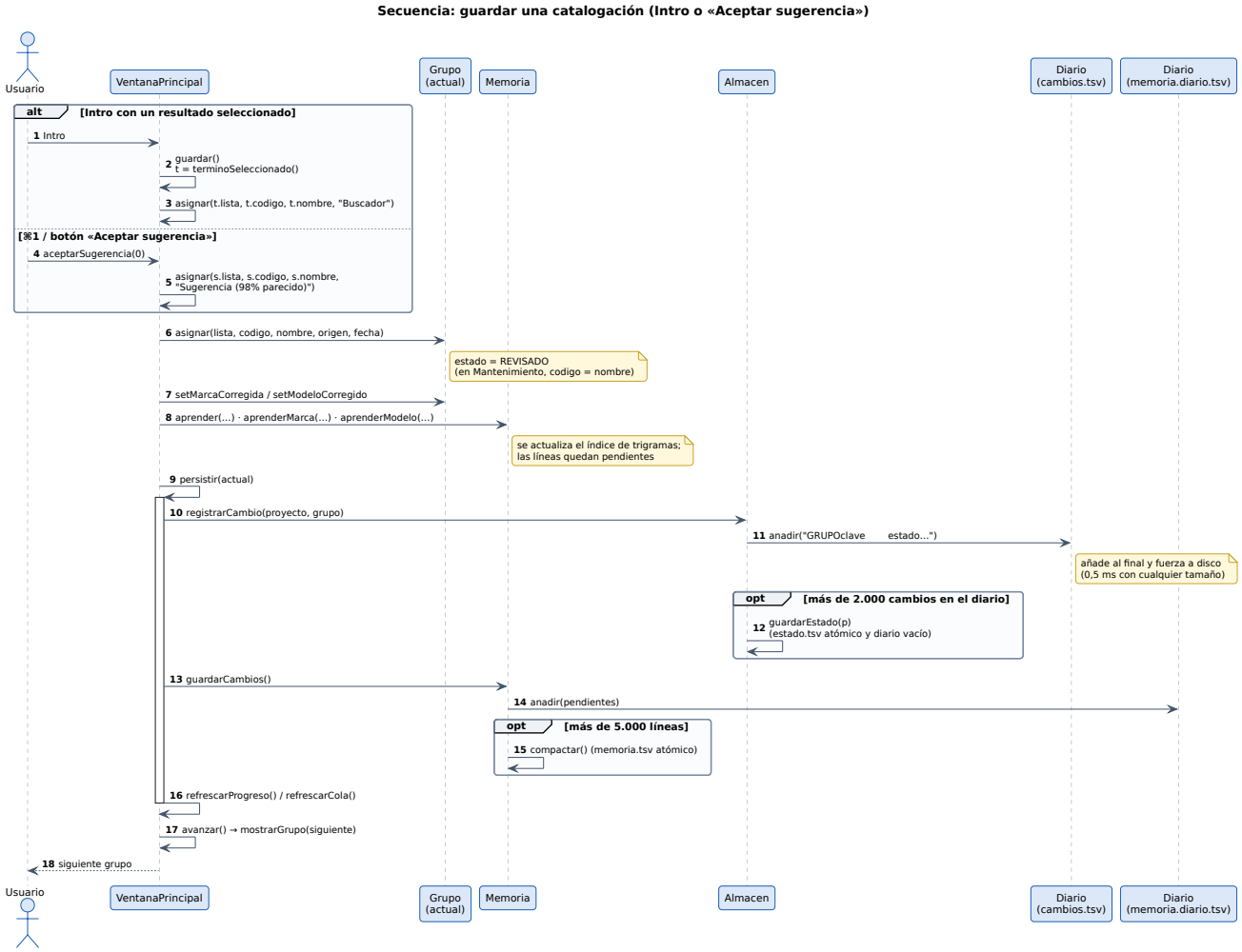


Figura 17. Secuencia: guardar una catalogación.

5.7 Crear una entrada de Mantenimiento

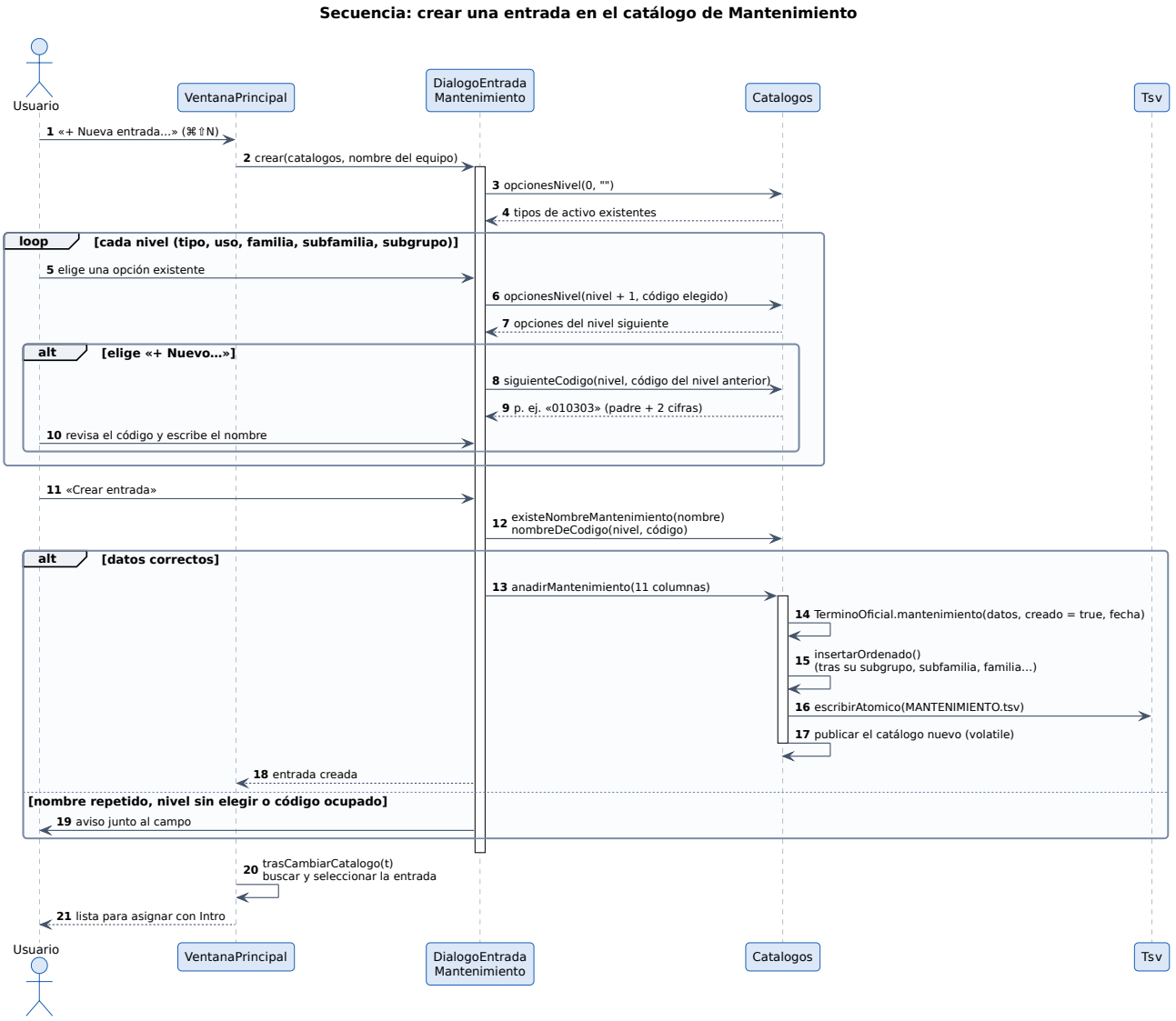


Figura 18. Secuencia: crear una entrada en el catálogo de Mantenimiento.

5.8 Exportar el resultado

El diálogo muestra los ficheros que se van a crear (con nombres únicos aunque dos centros solo se distingan por mayúsculas o por caracteres no válidos); luego se reparten los equipos por centro y cada fichero se escribe en flujo (cada fila se genera al escribirla) en un temporal que se renombra al terminar.

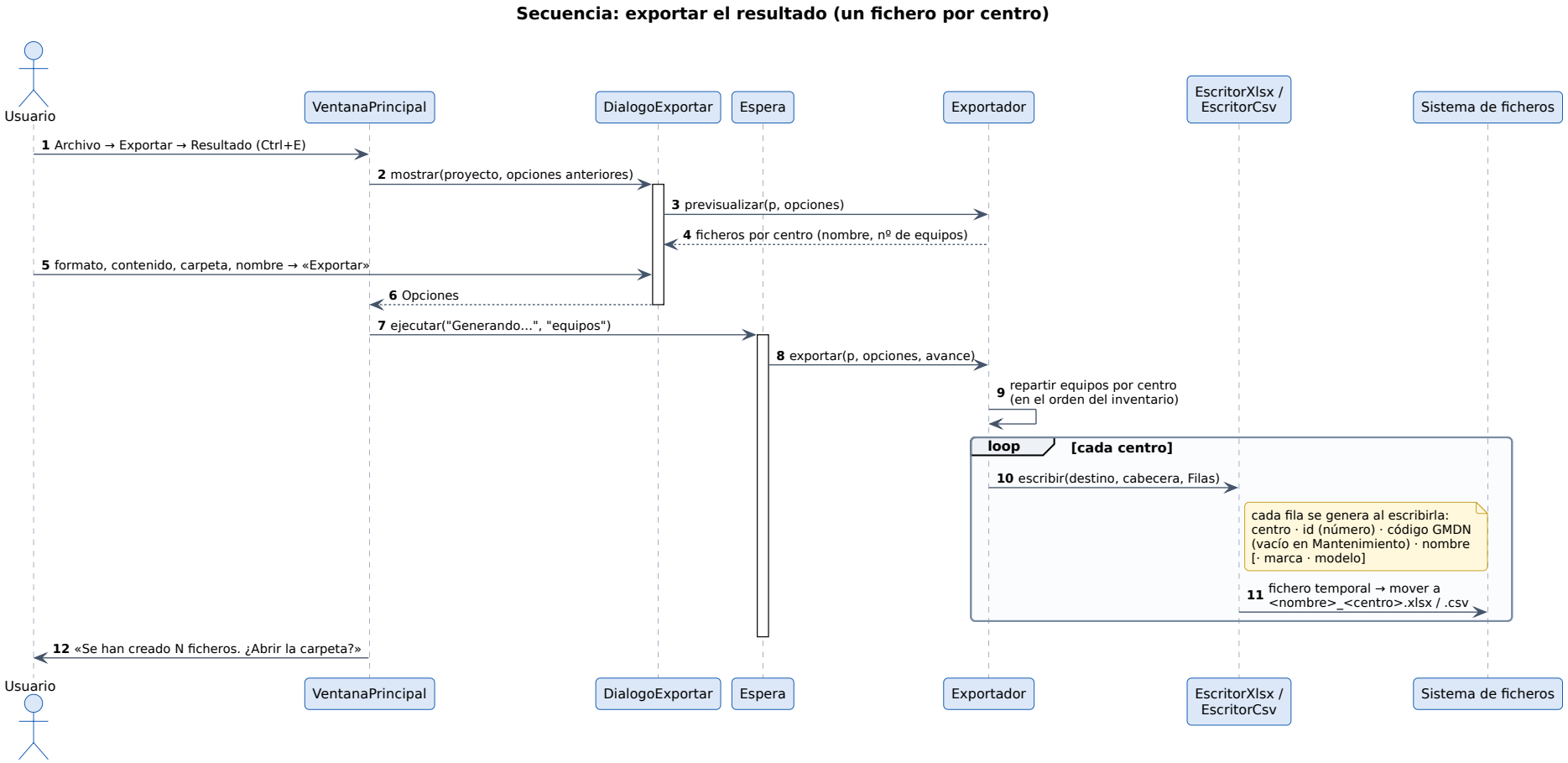


Figura 19. Secuencia: exportar el resultado.

5.9 Aplicar un resultado anterior

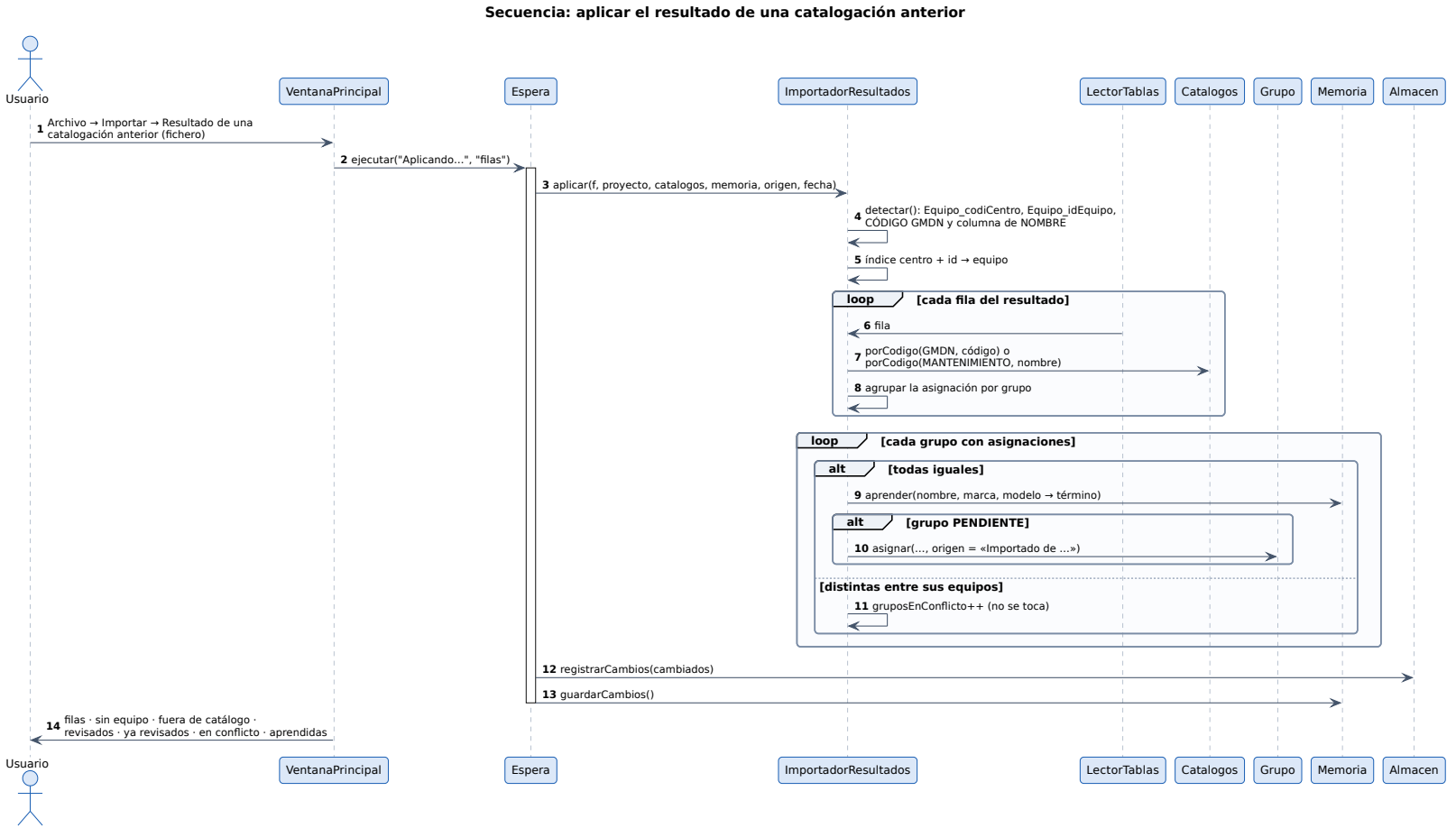


Figura 20. Secuencia: aplicar un resultado anterior.

5.10 Configurar el servicio de IA

El diálogo trabaja sobre la configuración en memoria y solo la guarda al aceptar; si se cancela, se vuelve a leer del disco. «¿Cómo se consigue?» muestra los pasos de `ProveedorIA.getPasos()`; «Cargar modelos» y «Probar conexión» se ejecutan en segundo plano.

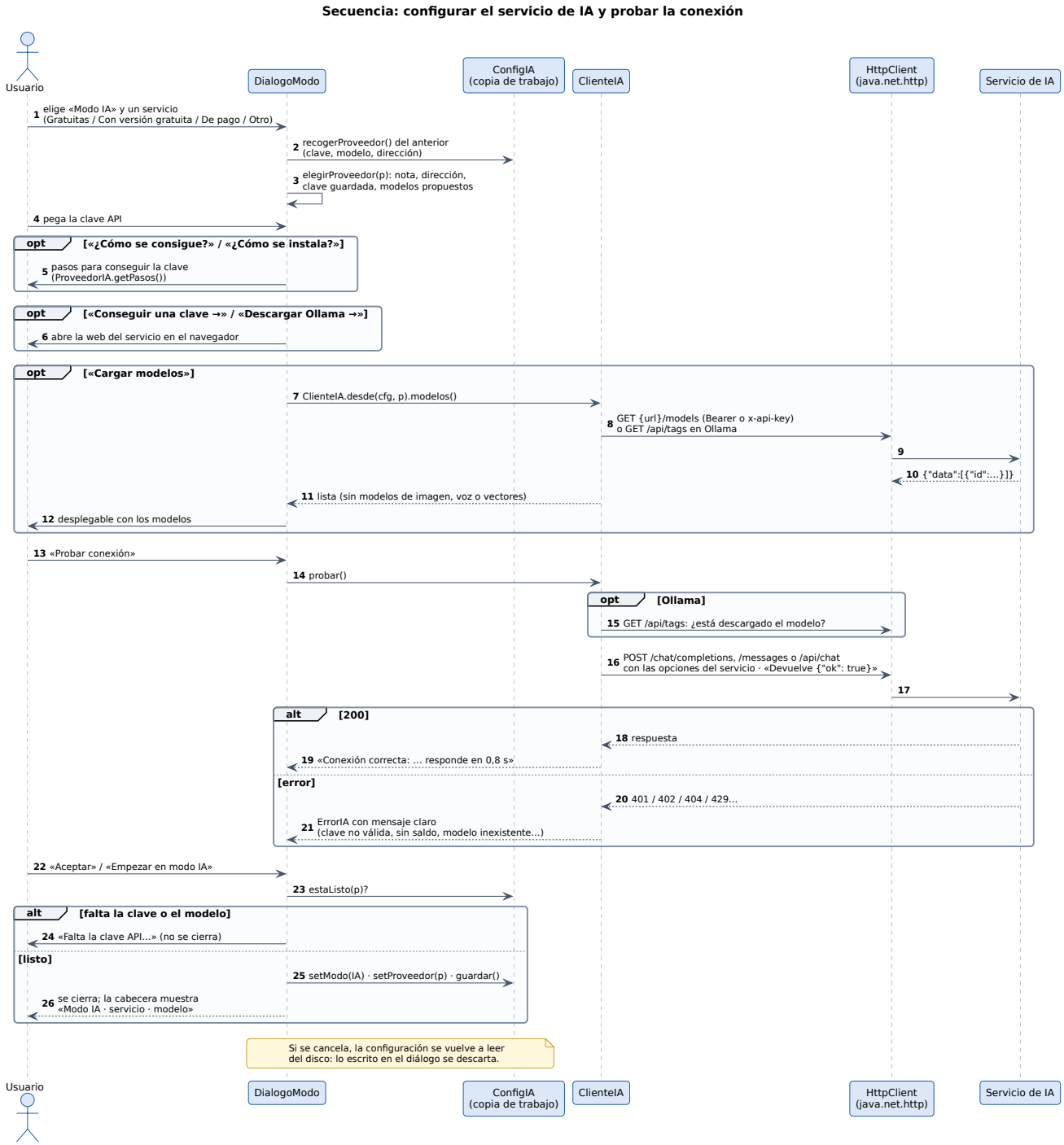


Figura 21. Secuencia: configurar el servicio de IA y probar la conexión.

5.11 Propuesta de la IA en segundo plano

Al mostrar un grupo, si no tiene propuesta, se pide junto con los siguientes pendientes de la cola (hasta el lote del servicio: 8, o 4 con Groq, Ollama y LM Studio). Si ya la tiene, solo se pide más cuando faltan al menos la mitad de los siguientes: así la IA va por delante de la persona sin hacer consultas de un solo grupo. Un único hilo «ia» hace las consultas en orden; la interfaz se actualiza con `invokeLater` solo si el lote incluye el grupo que se está viendo.

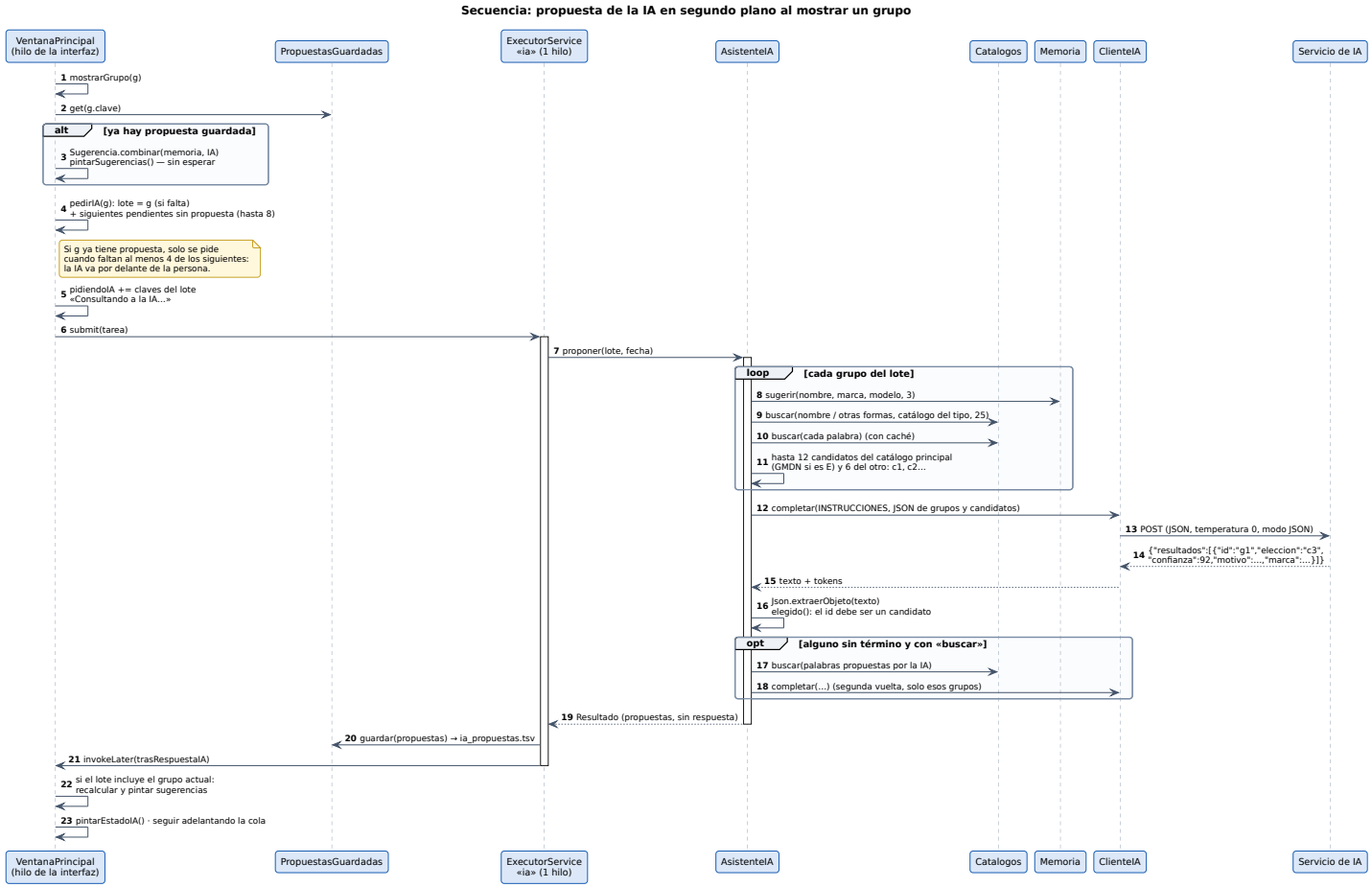


Figura 22. Secuencia: propuesta de la IA en segundo plano.

5.12 Una consulta a la IA: adaptación, reintentos y errores

`ClienteIA.completar` encapsula las diferencias entre servicios y los fallos de red: compone el cuerpo en el formato del servicio con las opciones que admite, quita y recuerda las que rechaza, respeta las esperas que pide el servicio (como mucho 90 s; más, es cuota agotada) y avisa si la respuesta se ha cortado, para que `AsistenteIA` parta el lote.

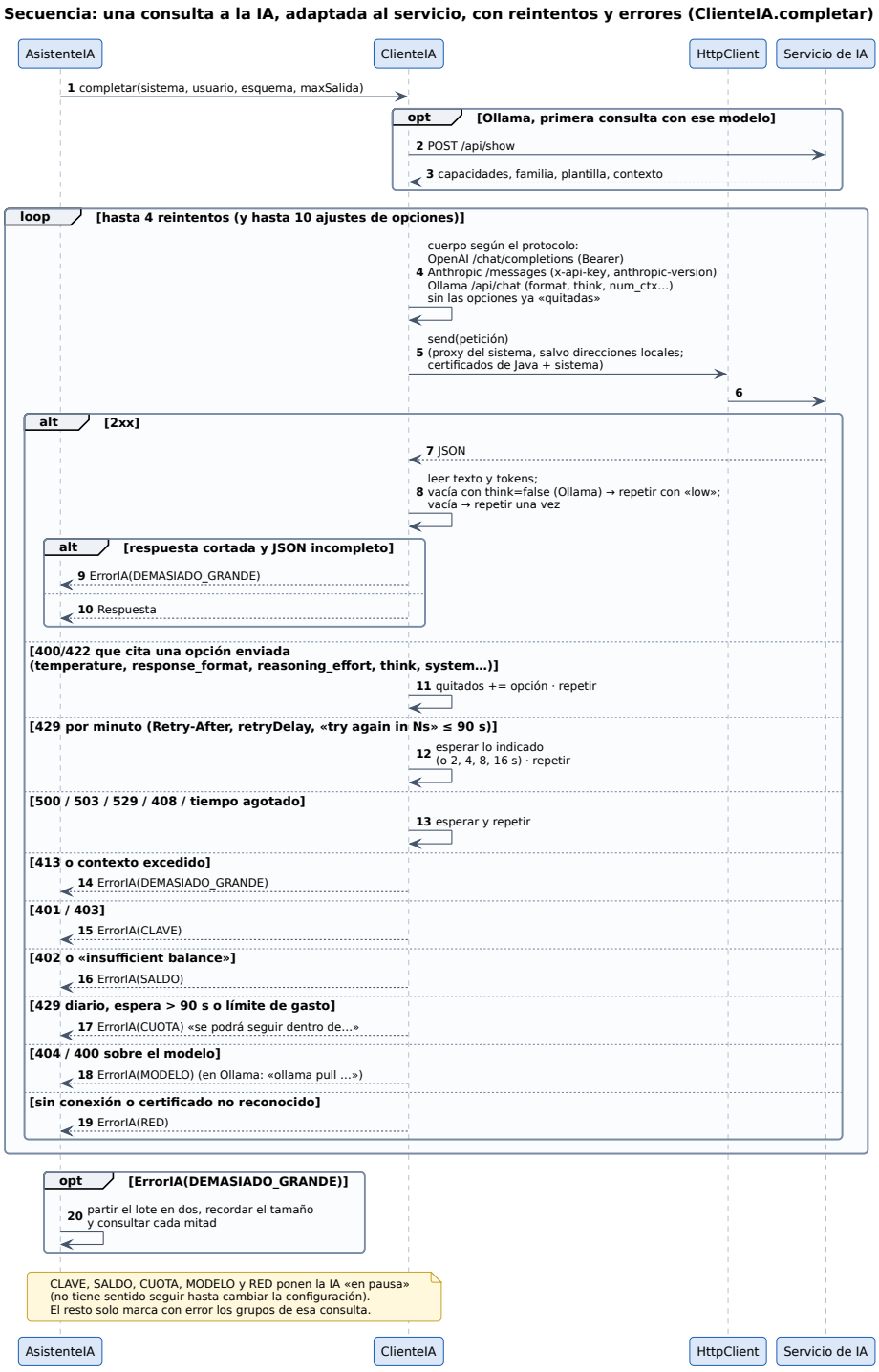


Figura 23. Secuencia: una consulta a la IA.

5.13 Pedir todas las propuestas y aceptar en bloque

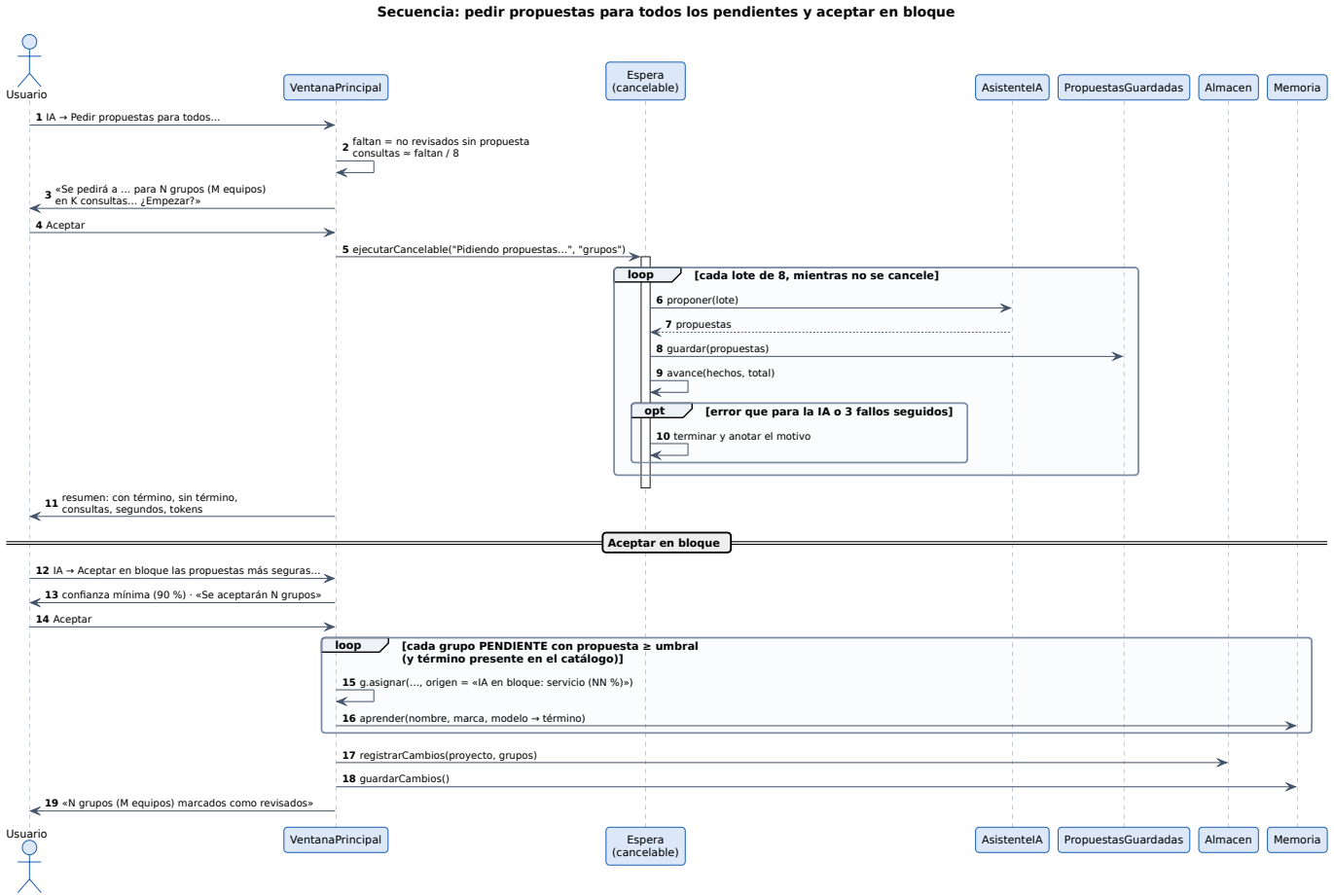


Figura 24. Secuencia: pedir propuestas para todos los pendientes y aceptar en bloque.

6 Diagramas de actividad (flujo)

6.1 Flujo de trabajo completo

Desde que se abre el programa hasta entregar el resultado. Las exportaciones se pueden hacer en cualquier momento.

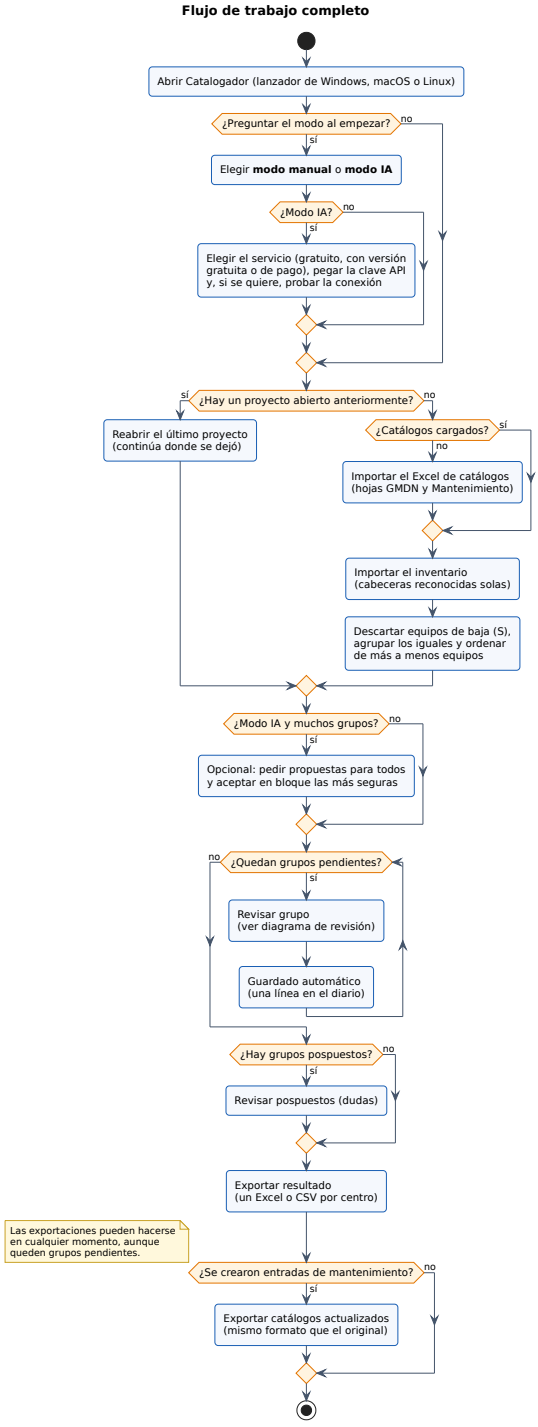


Figura 25. Flujo de trabajo completo.

6.2 Revisar un grupo

Con calles (swimlanes) para lo que hace la persona y lo que hace el sistema.

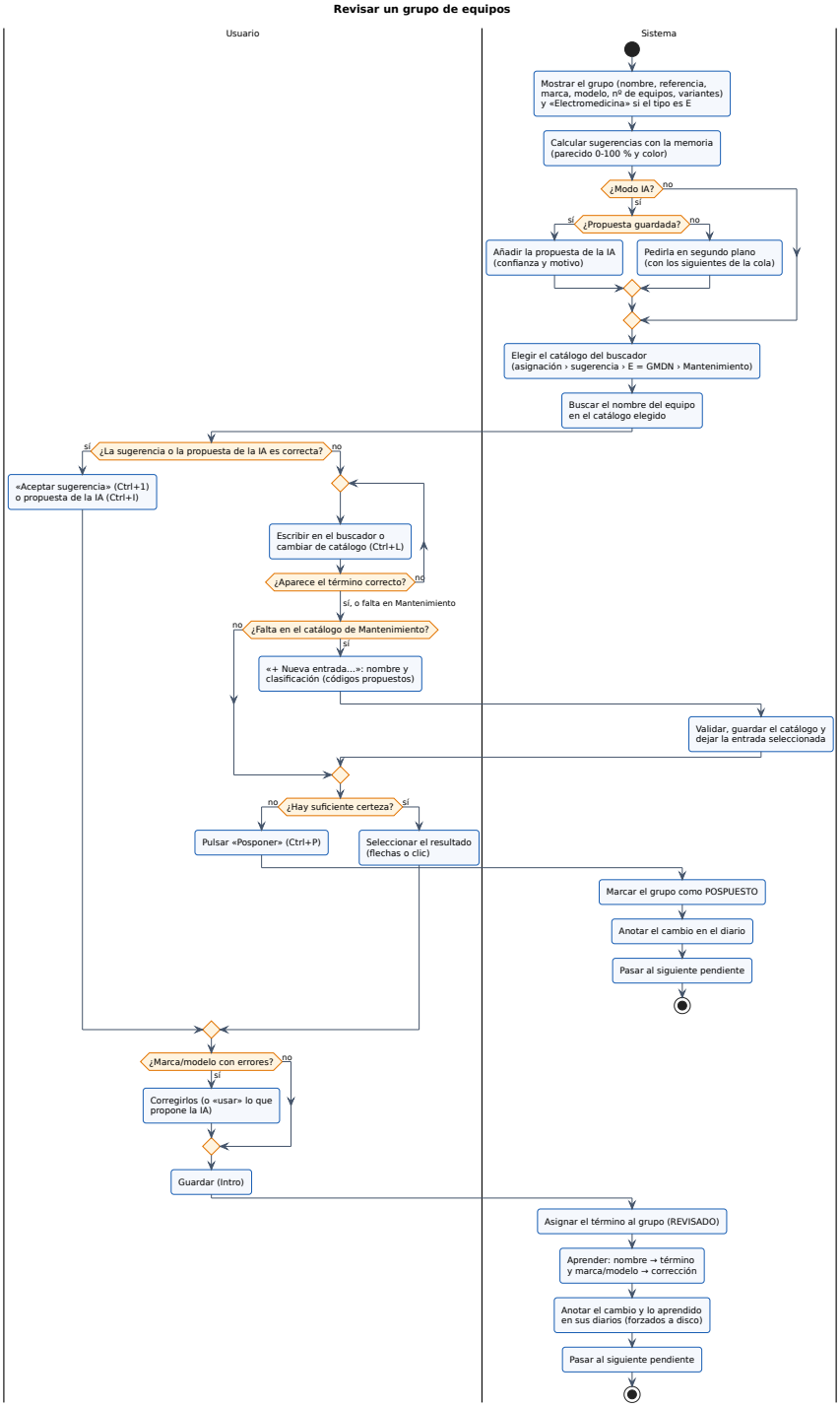


Figura 26. Actividad: revisar un grupo.

6.3 Importación fila a fila

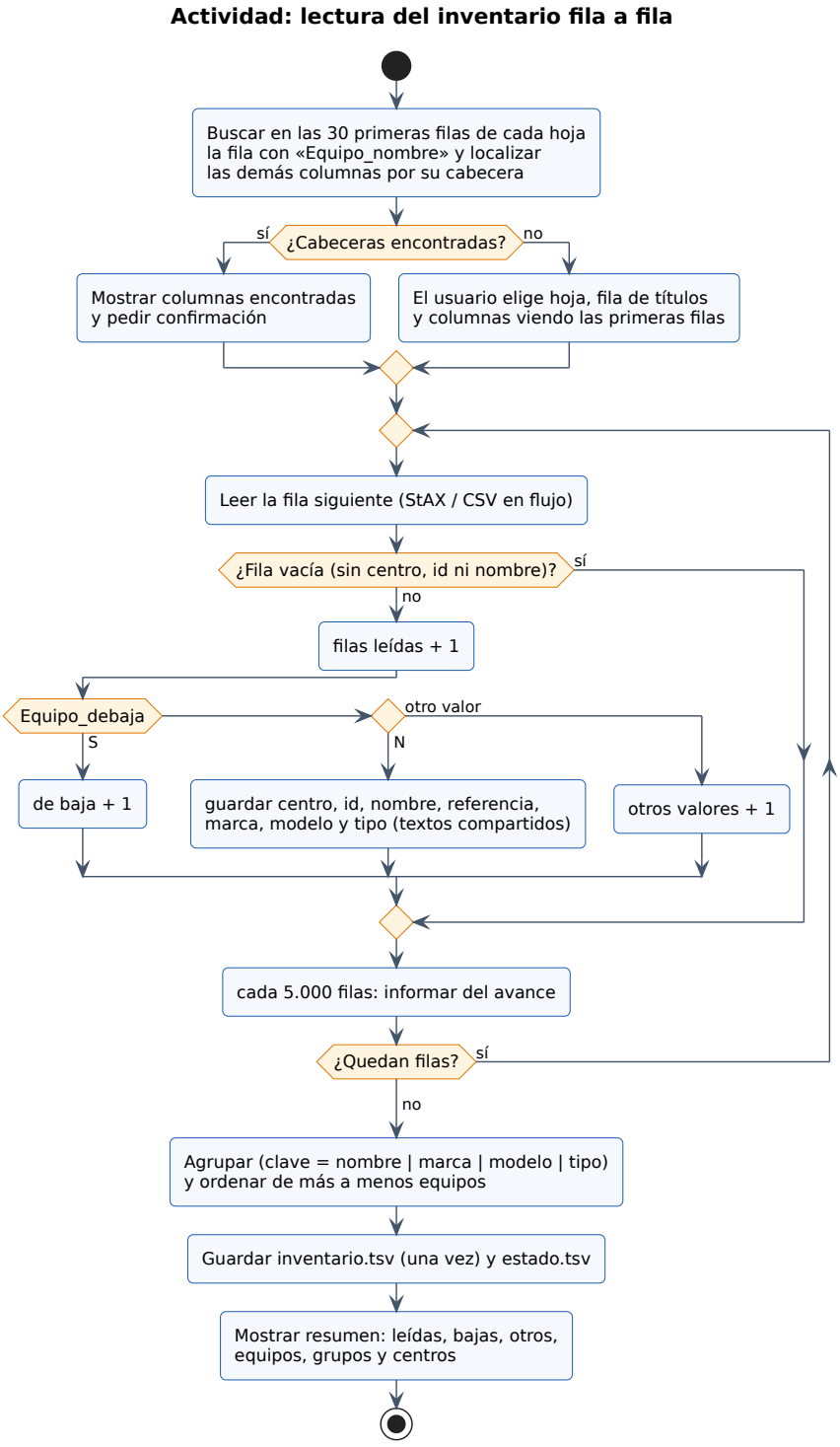


Figura 27. Actividad: importación del inventario en flujo.

6.4 Consulta a la IA de un lote de grupos

Tres fases: elección de candidatos (en el ordenador), consulta (adaptada al servicio, partiendo el lote si no cabe) y validación de la respuesta. Los grupos que la IA no contesta quedan para otra consulta.

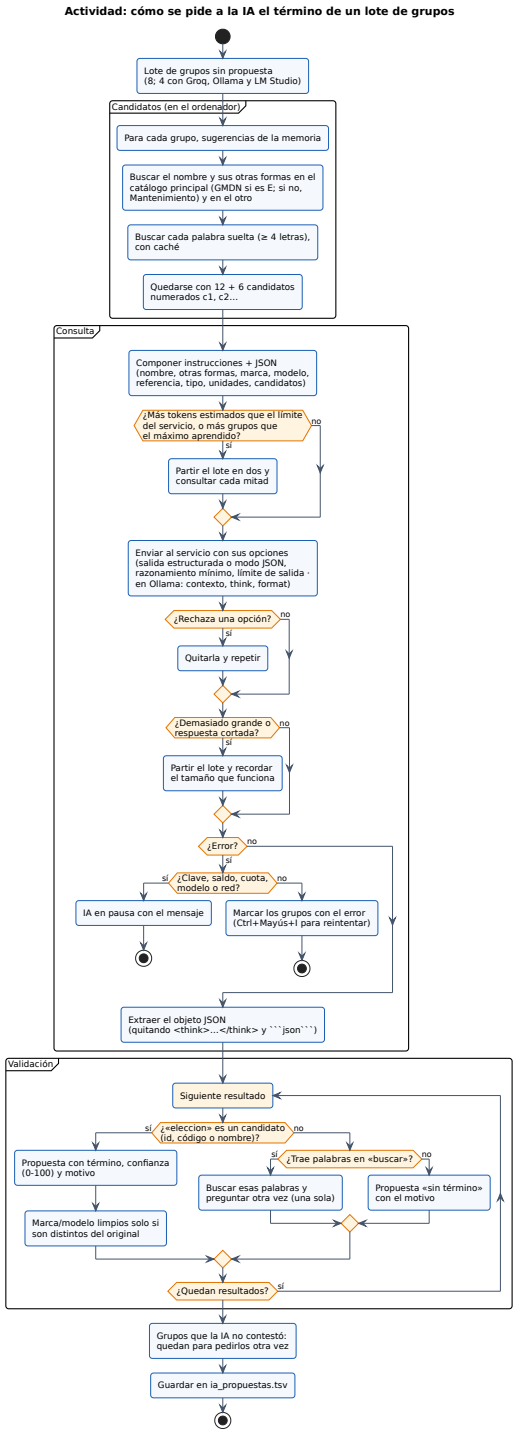


Figura 28. Actividad: consulta a la IA de un lote de grupos.

6.5 Guardado continuo y recuperación

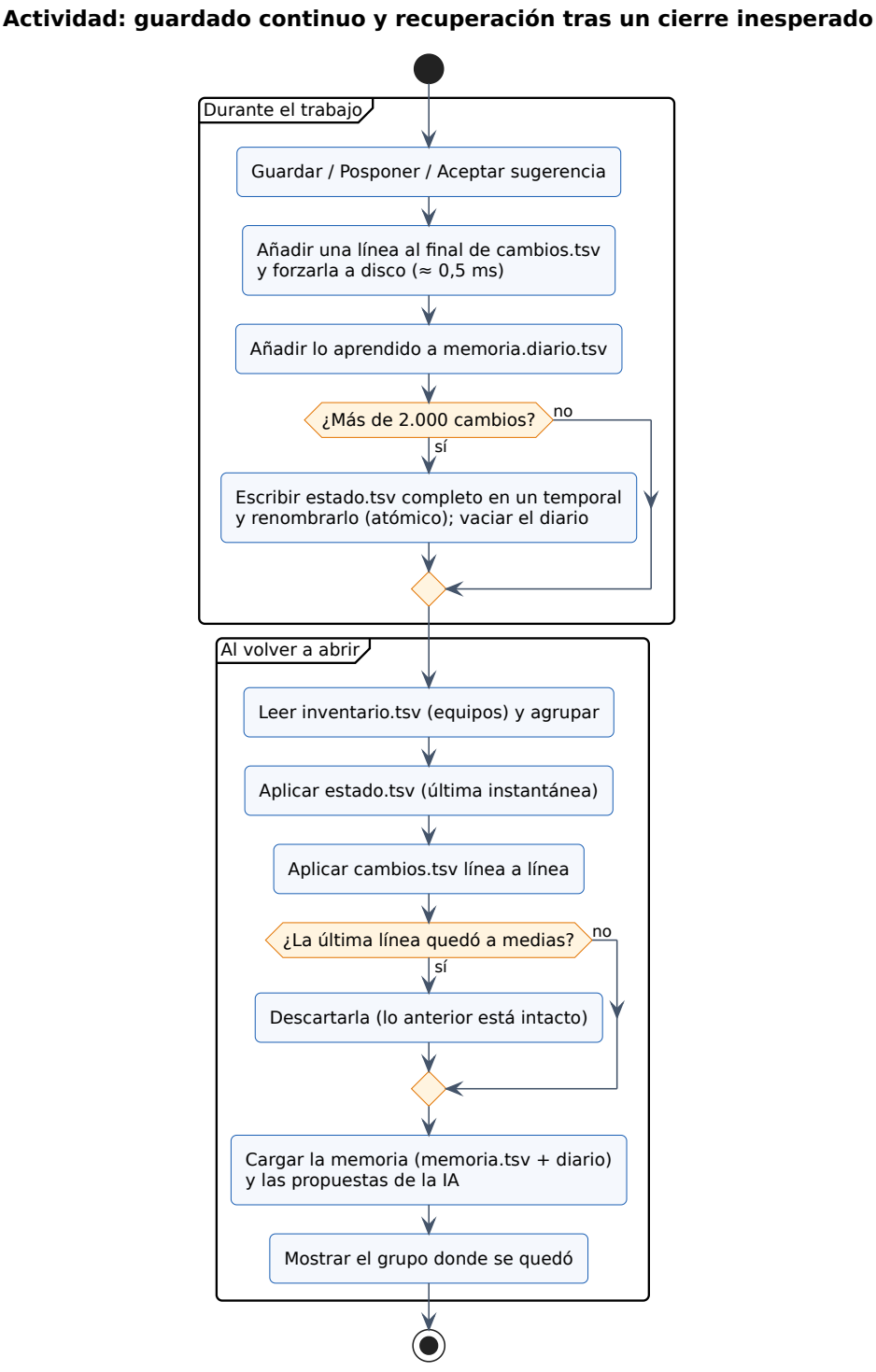


Figura 29. Actividad: guardado continuo y recuperación tras un cierre.

7 Diagramas de estados

7.1 Grupo de equipos



Figura 30. Estados de un grupo de equipos.

7.2 Aplicación

Incluye el estado concurrente del buscador (GMDN o Mantenimiento) y del modo de trabajo.

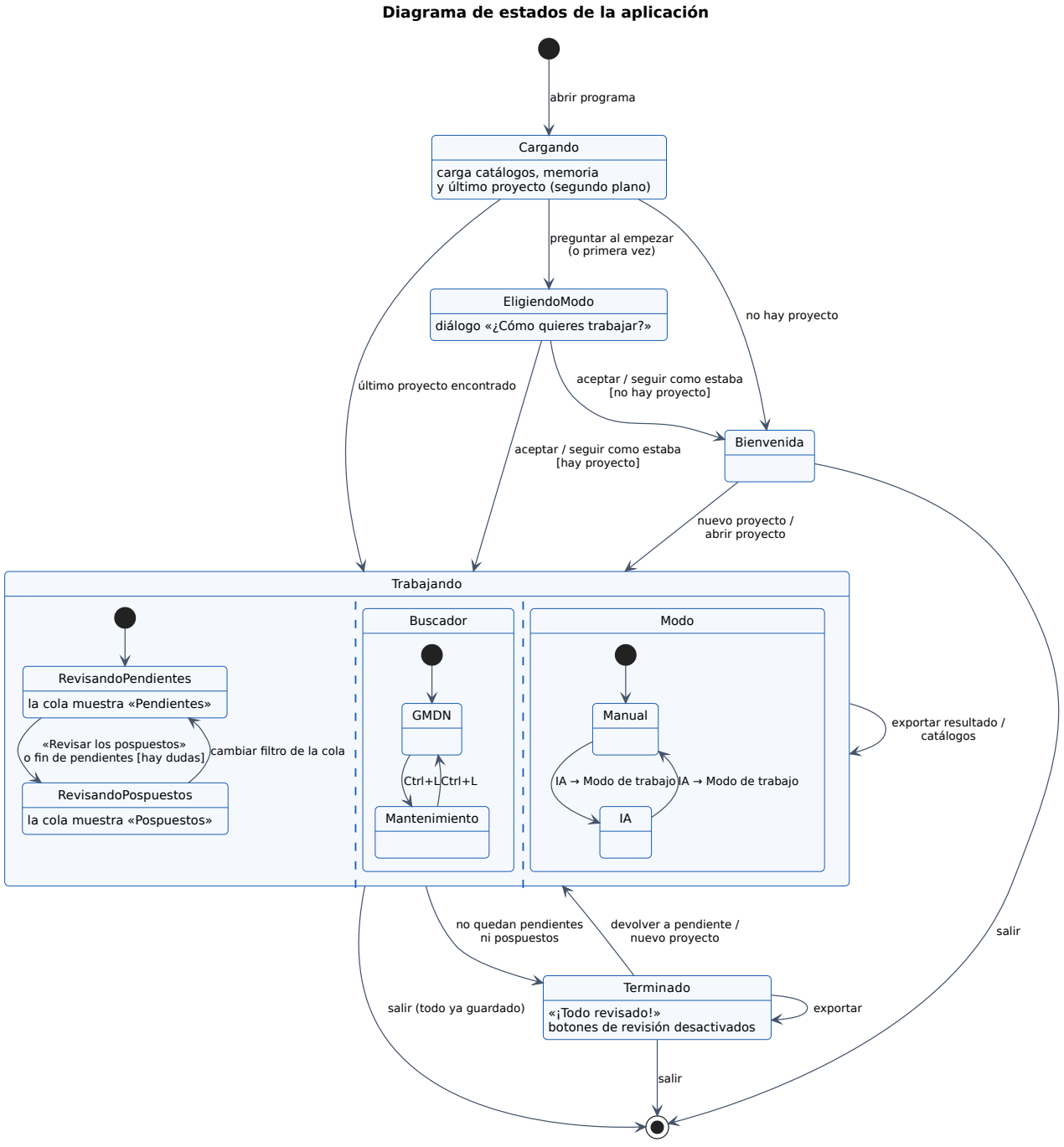


Figura 31. Estados de la aplicación.

7.3 Modo IA y propuesta de un grupo

Izquierda: el modo IA puede estar listo, consultando o en pausa (por clave, saldo, cuota, modelo o red). Derecha: la vida de la propuesta de un grupo, desde que se pide hasta que se acepta.

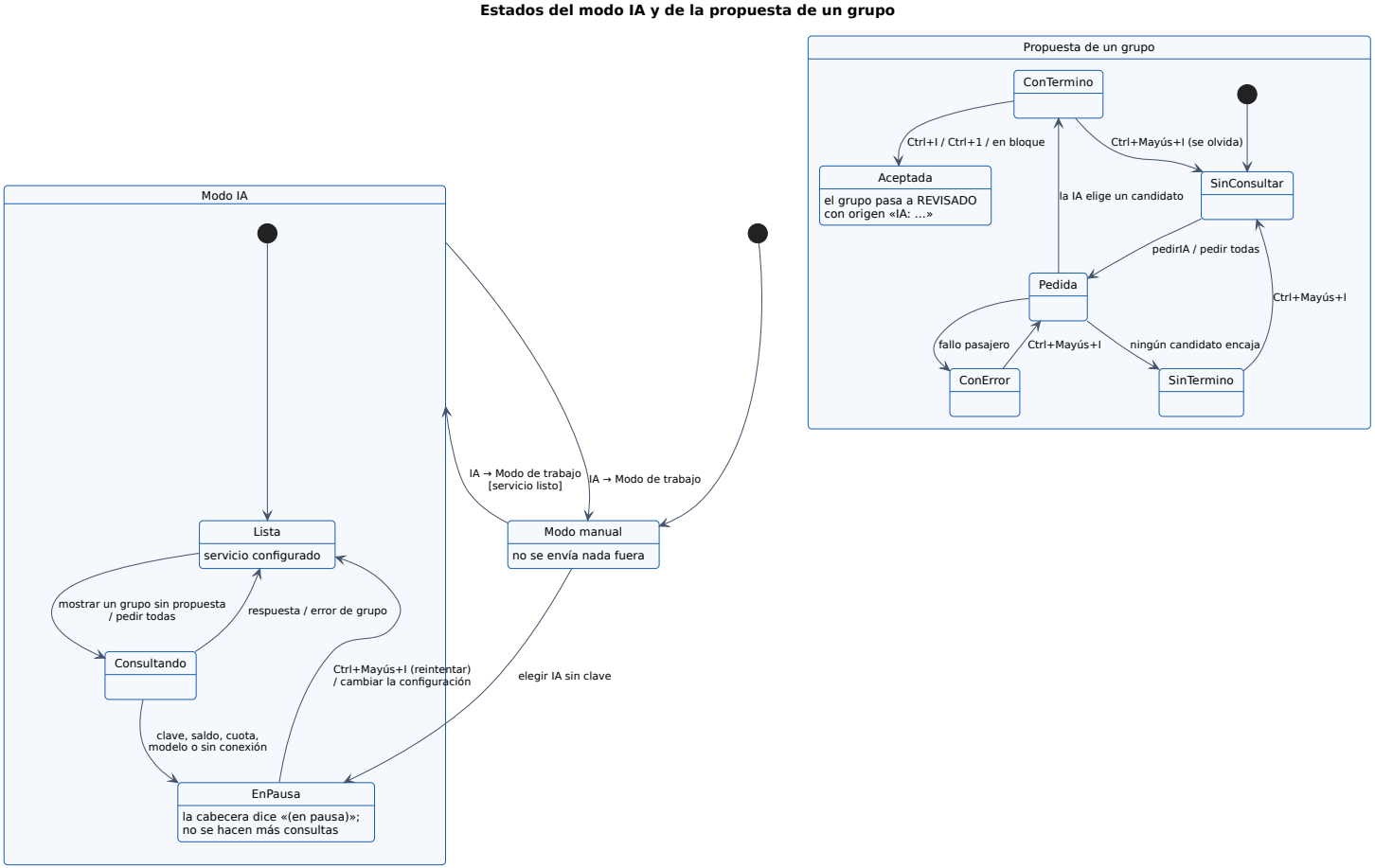


Figura 32. Estados del modo IA y de la propuesta de un grupo.

8 Datos

8.1 Ficheros que guarda el programa

Todos son de texto (UTF-8, campos separados por tabuladores y caracteres especiales escapados), salvo las configuraciones, que son ficheros `.properties` de Java.

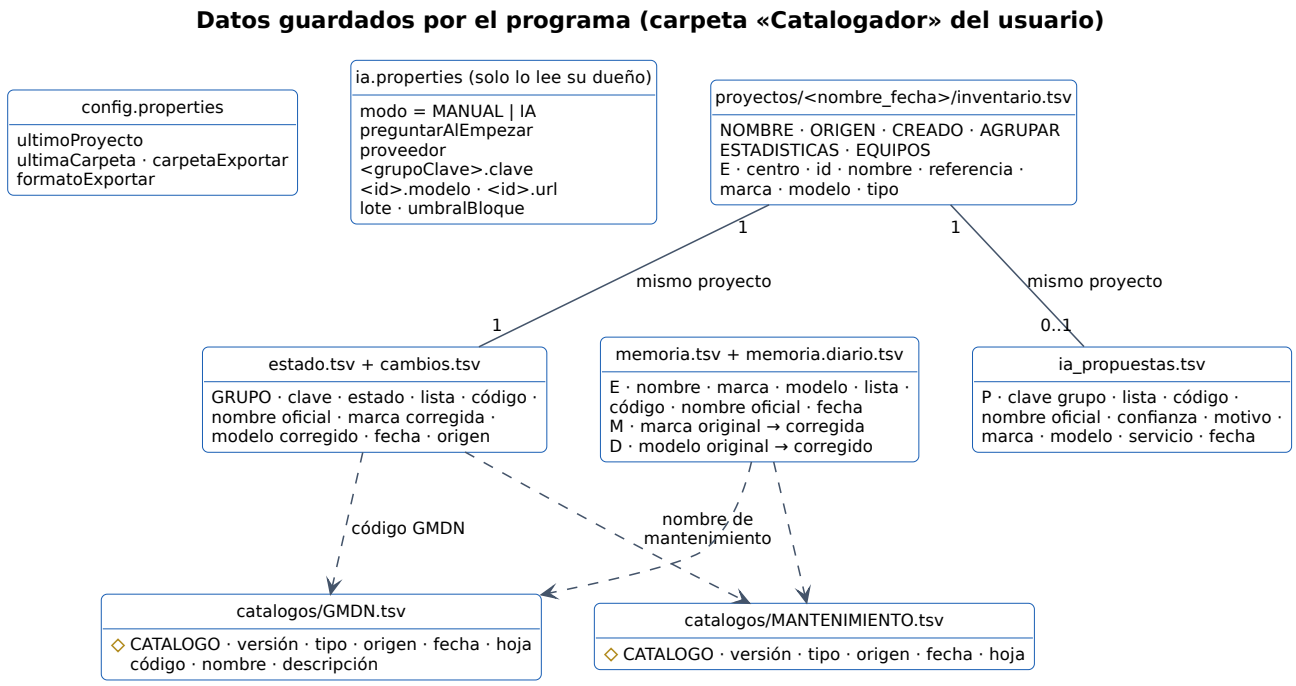


Figura 33. Ficheros en los que se guardan los datos.

Fichero	Cuándo se escribe
`catalogos/GMDN.tsv`, `MANTENIMIENTO.tsv`	Al importar un catálogo y al crear o corregir una entrada (escritura atómica).
`memoria.diario.tsv` → `memoria.tsv`	Cada clasificación aprendida se añade al diario; cada 5.000 líneas se compacta en memoria.tsv.
`proyectos/…/inventario.tsv`	Una vez, al crear el proyecto.
`proyectos/…/cambios.tsv` → `estado.tsv`	Cada decisión se añade a cambios.tsv; cada 2.000 cambios (y al abrir) se escribe estado.tsv y se vacía el diario.
`proyectos/…/ia_propuestas.tsv`	Cada lote de propuestas recibido (se reescribe al olvidar una propuesta).
`config.properties`, `ia.properties`	Al cambiar carpetas, último proyecto o la configuración de la IA (ia.properties, con permisos solo para su dueño en macOS y Linux).

9 Algoritmos

9.1 Agrupación

```
clave(equipo) = ordenar(palabrasSignificativas(normalizar(nombre)))
              + [ "|" + normalizar(marca) + "|" + normalizar(modelo) ]   // si se separa por marca y
modelo
              + "|" + tipoActivo
normalizar(s) = quitar tildes → minúsculas → todo lo que no es letra o cifra, espacio → recortar
grupos = equipos agrupados por clave, ordenados por número de equipos (descendente)
```

9.2 Similitud para las sugerencias

```
s = 0,55 · palabras(a, b) + 0,45 · dice(trigramas(a), trigramas(b))
palabras(a, b) = 2 · Σ parecido(x, mejor pareja libre de x en b) / (nº palabras de a + nº palabras
de b)
parecido(x, y) = 1 si son iguales · 0,8 si una es abreviatura de la otra (≥ 3 letras)
                 · 0,85 con una errata (dos si la palabra tiene más de 7 letras) · 0 si no
si s < 1: s += 0,03 si coincide la marca; s += 0,02 si coincide el modelo (máximo 0,99)
si s = 1 y cambian marca o modelo: s = 0,98
se proponen las 3 mejores con s ≥ 0,45 (una por término); color: ≥ 0,85 verde, ≥ 0,65 naranja, resto
rojo
```

9.3 Puntuación del buscador

```
para cada término del catálogo elegido:
   para cada palabra p de la consulta:
       3 si p es una palabra del nombre · 2,5 si es su comienzo · 1,5 si está dentro · 1,2 con una
errata
       (GMDN: 6 si p es el código, 4 si es su comienzo)
       si no está en el nombre y se busca en la descripción: cuenta como «en descripción»
   total = 10·(palabras en el nombre) + puntos + 3·(palabras solo en la descripción) − 0,01·longitud
del nombre
           + 6 si el nombre contiene la consulta entera + 20 si es igual
           + 100 si el código es igual a la consulta + 40 si empieza por ella
se ordena por total (máximo 300 resultados)
```

9.4 Candidatos para la IA

```
principal = GMDN si el grupo es de electromedicina (E); si no, Mantenimiento
candidatos = términos de las sugerencias de la memoria (si están en el catálogo cargado)
para cada catálogo (principal con cupo 12, el otro con cupo 6; +6 en la segunda vuelta):
   sumar puntuaciones de: buscar(nombre) · buscar(otras formas distintas, ×0,8) · buscar(palabras
propuestas, ×1,2)
                          · buscar(cada palabra de 4 o más letras, ×0,35, con caché)
   si hay menos de 3: buscar también en la descripción (×0,5)
   añadir los mejores hasta el cupo (sin repetir)
numerarlos c1, c2, …
```

9.5 Tamaño de cada consulta a la IA

```
lote = «lote» de ia.properties, o el del servicio (8; 4 en Groq, Ollama y LM Studio)
tokens estimados = (instrucciones + datos en JSON) / 3
si tokens estimados > límite del servicio (12.000; 4.500 en Groq; 7.000 en los locales)
   o hay más grupos que el máximo aprendido:          partir en dos mitades (recursivamente)
si el servicio responde «demasiado grande» o la respuesta llega cortada:
   máximo aprendido = mitad del lote; partir; un grupo solo se envía en forma compacta
   (8 candidatos y descripciones de 100 caracteres)
Ollama: num_ctx = redondear_a_2048(tokens estimados + 64 + salida + 256), entre 8.192 y 32.768
```

10 Pruebas

Las pruebas automáticas están en el código fuente (`codigo-fuente/pruebas`) y se describen, con sus entradas y resultados, en la ERS (capítulo 10). Su relación con el diseño:

Prueba	Clases que ejercita
PruebaRequisitos	`ImportadorInventario`, `ImportadorCatalogos`, `ImportadorResultados`, `Catalogos`, `Memoria`, `Grupo`, `Almacen`, `Exportador`, `ExportadorCatalogo`, `DialogoImportacion.adivinar`, con los Excel recibidos (incluido el inventario recibido catalogado y comparado fila a fila), el paquete de pruebas y el inventario de 600.000 filas.
PruebaIA	`Json`, `ProveedorIA`, `ConfigIA`, `ClienteIA`, `AsistenteIA`, `PropuestasGuardadas`, `Sugerencia.combinar` — contra un servidor simulado (`mock_ia.py`) que reproduce la API y el comportamiento documentado de cada servicio (OpenAI, Gemini, Anthropic, Groq, Mistral, DeepSeek, OpenRouter, Cerebras, Cohere, xAI, Hugging Face, LM Studio y Ollama con diez modelos) y sus errores.
Rendimiento	Todo el camino de datos con 650.000 y 1.000.000 de filas, con comparación fila a fila del resultado exportado; el modo IA con los formatos OpenAI y Ollama.
CapturasManual	La interfaz completa (`VentanaPrincipal` y todos los diálogos) recorrida como una persona, en modo manual y en modo IA, con servidores de IA simulados.